

RTL-to-GDS Implementation

8-bit Arithmetic Logic Unit (ALU)

Cadence Digital Implementation Flow on 90nm Technology

Complete Project Handbook and Interview Preparation Guide

Tools: NCVerilog | Cadence Genus | Cadence Tempus | Cadence Conformal LEC | Cadence Innovus
Technology Library: 90nm Cadence Generic Standard Cell Library (slow corner)

June 2026

Table of Contents

Table of Contents	2
1. Project Overview	7
2. Objective and Problem Statement	8
2.1 Objective	8
2.2 Problem Statement	8
2.3 Specifications Summary	8
3. ASIC Design Flow Overview	9
3.1 Flow Stages	9
4. Complete Toolchain Overview	10
4.1 NCVerilog (Cadence Xcelium)	10
4.2 Cadence Genus	10
4.3 Cadence Tempus	11
4.4 Cadence Conformal LEC	12
4.5 Cadence Innovus	12
5. RTL Design	13
5.1 Module Interface	13
5.2 Internal Registers	13
5.3 Operation Encoding	13
5.4 Synthesizability Analysis	14
5.5 Hardware Generated by Synthesis	14
6. Simulation and Testbenches	15
6.1 Simulation Flow	15
6.2 Testbench 1 (tb1.v) - Basic Functional Verification	15
6.3 Testbench 2 (tb2.v) - Corner Case Testing	15
6.4 Testbench 3 (tb3.v) - Extended Coverage	15
7. Coverage Analysis	16
7.1 What is Coverage?	16
7.2 Coverage Types	16
7.3 How Coverage Was Improved	16
8. Constraints (SDC)	17
8.1 Constraint Comparison	17
8.2 Detailed Constraint Explanation	17
8.2.1 create_clock	17
8.2.2 set_clock_transition	17
8.2.3 set_clock_uncertainty	17
8.2.4 set_input_transition	18

8.2.5 set_load	18
8.3 Constraints Not Used (But Important to Know)	18
9. Synthesis	19
9.1 Synthesis Flow Overview	19
9.2 Why Three Netlists?	19
9.3 Synthesis Results Comparison	19
9.4 Analysis of Results	19
10. Logic Equivalence Checking (LEC)	21
10.1 What is LEC?	21
10.2 LEC Flow	21
10.3 LEC Results - Good Netlists	21
10.4 LEC Results - Bad Netlists	21
10.4.1 Bad1 Analysis: Missing Flip-Flop	21
10.4.2 Bad2 Analysis: Miswired Net	21
10.4.3 Bad3 Analysis: Wrong Cell Type	22
11. Static Timing Analysis (STA)	23
11.1 STA Fundamentals	23
11.1.1 Key Timing Formulas	23
11.2 STA Strategy in This Project	23
11.3 STA Results Comparison (GBA Setup Analysis at 8 ns Period)	23
11.4 Why Min Area Netlist Fails at 8 ns	23
11.5 Why Tight Timing Has the Most Slack at 8 ns	23
11.6 Hold Timing Analysis	23
11.7 STA Parametric Experiments	24
12. Design for Testability (DFT)	25
12.1 Why DFT is Required	25
12.2 Scan Chain Architecture	25
12.3 Scan Chain Order	25
12.4 How Muxed Scan Works	25
12.5 DFT Impact on Area and Timing	25
13. Post-DFT Static Timing Analysis	27
13.1 Why Post-DFT STA is Required	27
13.2 Post-DFT STA Configuration	27
13.3 Post-DFT Timing Results	27
14. Physical Design	28
14.1 MMMC Setup	28
14.2 Floorplanning	28

14.2.1 Die Size Calculation	28
14.2.2 I/O Pin Placement	28
14.2.3 Power Ring	28
14.3 Placement.....	29
14.3.1 Post-Placement Timing	29
14.4 Clock Tree Synthesis (CTS)	29
14.4.1 CTS Buffer Cells	29
14.4.2 Post-CTS Timing.....	29
14.5 Routing	29
14.5.1 Post-Route Timing.....	30
14.5.2 Post-Route Area	30
14.6 Signoff.....	30
14.6.1 DRC Results	30
14.6.2 Connectivity Results.....	30
14.6.3 Final Outputs.....	30
15. Complete Timing Progression Summary	31
16. Final Design Summary.....	32
17. Complete Interview Preparation	33
18. Appendix	44
18.1 Glossary	44
18.2 Important Formulas	44
18.3 File Structure	44
18.4 Key Cadence Commands Reference	45
19. Detailed Script Analysis.....	46
19.1 Synthesis Script Deep Dive (synth_balanced.tcl)	46
19.1.1 Directory Creation.....	46
19.1.2 Search Path Configuration	46
19.1.3 Library Loading.....	46
19.1.4 Write SDF Options.....	46
19.2 LEC Script Deep Dive (lec_balanced.do)	47
19.2.1 Library and Design Loading.....	47
19.2.2 Comparison Execution	47
19.3 STA Script Deep Dive (sta_min_area.tcl).....	47
19.3.1 Why All Three Netlists Use the 8 ns Constraint	47
19.3.2 PBA vs GBA Explanation.....	47
19.3.3 check_timing Report Analysis.....	48
19.4 DFT Script Deep Dive (dft_balanced.tcl).....	48

19.4.1 DFT Configuration	48
19.4.2 DFT Rule Checking and Violation Fixing	48
19.4.3 Scan Chain Connection.....	48
19.5 Innovus Floorplan Script Deep Dive (floorplan_05.tcl)	49
19.5.1 Design Initialization Variables	49
19.5.2 Power Ring Parameters Explained	49
19.6 Innovus Placement Script Deep Dive (placement.tcl)	49
19.6.1 Scan Chain Definition Before Placement.....	50
19.6.2 Post-Placement Analysis Commands	50
19.7 Innovus CTS Script Deep Dive (cts.tcl)	50
19.7.1 Buffer Cell Selection.....	50
19.7.2 Clock Tree Specification	50
19.8 Innovus Routing Script Deep Dive (routing.tcl)	50
19.8.1 NanoRoute Configuration.....	50
20. Detailed Report Analysis.....	52
20.1 Synthesis Area Report Breakdown	52
20.2 Cell Count Analysis	52
20.3 Power Report Analysis.....	52
20.4 Timing Report Reading Guide	53
20.5 Physical Design Area Progression.....	53
20.6 Physical Design Timing Progression (Hold).....	54
21. Physical Design Detailed Walkthrough.....	55
21.1 Metal Layer Usage.....	55
21.2 Design Rule Violations Analysis.....	55
21.3 Connectivity Verification	55
21.4 Gate Count Summary	55
21.5 Supply Voltage	56
22. STA Experiments Deep Dive.....	57
22.1 Clock Slew Sweep (Min Area Netlist at 8 ns).....	57
22.2 Output Load Sweep (Min Area Netlist at 8 ns).....	57
22.3 Post-DFT Slew and Load Matrix.....	57
23. Industry Best Practices vs. This Project	58
24. Common Interview Mistakes to Avoid	59
24.1 Technical Mistakes	59
24.2 Presentation Mistakes	59
24.3 Conceptual Mistakes.....	59
25. Quick Reference Cards	60

25.1 Numbers You Must Know	60
25.2 Tool Command Quick Reference	60
25.3 Flow Dependency Chain.....	60
25.4 Critical Path Summary.....	61

1. Project Overview

This document is a complete handbook for an RTL-to-GDS implementation of an 8-bit Arithmetic Logic Unit (ALU) using the Cadence digital implementation flow on a 90nm generic standard cell library. The project traverses every stage of the ASIC design flow, from writing the Verilog RTL code, through functional simulation, code coverage analysis, logic synthesis with three different optimization strategies, formal equivalence checking (LEC), static timing analysis (STA), design-for-testability (DFT) scan insertion, post-DFT STA, and finally complete physical design in Cadence Innovus including floorplanning, power planning, placement, clock tree synthesis (CTS), routing, and signoff with GDSII stream-out.

The ALU supports 16 arithmetic and logical operations selected by a 4-bit control input (sel). It takes two 8-bit input operands (in1, in2) and produces two 8-bit outputs (out1, out2). The design is fully synchronous, clocked by a single-phase rising-edge clock. All inputs are registered into internal flip-flops (X, Y, sel_ff), and the operation result appears on the outputs one clock cycle after the inputs change. This pipeline-register architecture creates a one-cycle latency, a critical interview point.

Three synthesized netlists were generated from the same RTL but with different SDC constraints, producing a Minimum Area netlist (15 ns clock, 66.67 MHz), a Balanced netlist (8 ns clock, 125 MHz), and a Tight Timing netlist (4 ns clock, 250 MHz). All three passed formal equivalence checking against the golden RTL. The Balanced netlist was carried forward through DFT scan insertion and the complete physical design flow.

The final physical design was completed at 50% utilization with a die area of approximately 103.24 um x 99.47 um. The design achieved timing closure with a post-route setup slack of +59 ps and zero hold violations after CTS optimization. The final GDSII was streamed out successfully with 12 DRC violations (8 outOfDie + 4 Metal1 shorts from power ring edge effects) and zero connectivity violations.

2. Objective and Problem Statement

2.1 Objective

To implement a complete RTL-to-GDS flow for an 8-bit ALU using the Cadence digital design toolchain, demonstrating industry-standard ASIC design methodology at every stage. The project aims to produce a manufacturable GDSII layout starting from behavioral Verilog RTL code.

2.2 Problem Statement

Design and implement an 8-bit ALU that supports 16 operations (ADD, SUB, MUL, DIV, MOD, AND, NAND, OR, NOR, XOR, XNOR, Vector Reverse, Parity, Sign Detection, Bitwise Equality, Average). The design must be fully synchronous with a single clock domain. Three synthesis strategies must be explored (minimum area, balanced, tight timing) to understand the area-timing-power trade-off. The balanced netlist must be carried through DFT insertion and complete physical design to GDSII tape-out.

2.3 Specifications Summary

Parameter	Value
Data Width	8-bit inputs, 8-bit dual outputs
Operations	16 (sel = 4'b0000 to 4'b1111)
Clock	Single-phase, rising-edge triggered
Reset	Active-high synchronous reset
Pipeline Latency	1 clock cycle (inputs registered)
Technology	90nm Cadence generic standard cell library
Target Frequency (Balanced)	125 MHz (8 ns period)
Target Utilization (PD)	50%
Final Die Size	103.24 um x 99.47 um

3. ASIC Design Flow Overview

The ASIC (Application-Specific Integrated Circuit) design flow is the sequence of steps that transforms a design idea into a physical chip layout ready for manufacturing. Each step feeds its output into the next step. If any step fails verification, the designer must go back and fix the issue before proceeding.

3.1 Flow Stages

The complete flow used in this project is:

Stage	Description
1. RTL Design	Write the hardware description in Verilog. This describes WHAT the circuit does.
2. Simulation	Verify functional correctness by applying test inputs and checking outputs.
3. Coverage Analysis	Measure how thoroughly the testbench exercises the RTL code.
4. Synthesis	Convert RTL into a netlist of standard cells (gates and flip-flops) from a technology library.
5. LEC	Formally prove the synthesized netlist is logically equivalent to the original RTL.
6. STA	Analyze all timing paths to verify setup and hold constraints are met without running simulation.
7. DFT	Insert scan chains to enable manufacturing test (stuck-at fault detection).
8. Post-DFT STA	Re-verify timing after scan insertion because scan muxes add delay.
9. Physical Design	Place cells, build clock tree, route wires, and optimize for timing/DRC closure.
10. Signoff	Final DRC/connectivity checks and GDSII stream-out for fabrication.

4. Complete Toolchain Overview

This chapter explains every Cadence tool used in this project. For each tool, we describe its purpose, what it takes as input, what it produces as output, key commands, and its role in industry.

4.1 NCVerilog (Cadence Xcelium)

NCVerilog is Cadence's Verilog simulator. It compiles and simulates Verilog RTL and testbenches. In this project, NCVerilog was used for functional simulation and coverage collection.

Purpose:

Run functional simulation of the ALU with testbenches and collect code coverage metrics (line, toggle, branch, expression coverage).

Inputs:

- RTL source file: rtl/ALU.v
- Testbench file: testbench/tb1.v, tb2.v, or tb3.v

Outputs:

- VCD waveform dump (tb1.vcd, tb2.vcd, tb3.vcd)
- Coverage database (used by IMC for coverage analysis)
- Simulation pass/fail console output

Key Command:

```
ncverilog rtl/ALU.v testbench/tb3.v +access+r +gui -coverage all -covoverwrite
```

Breakdown of this command:

Option	Meaning
ncverilog	Invokes the Cadence Verilog simulator
rtl/ALU.v	Path to the RTL design source file
testbench/tb3.v	Path to the testbench file
+access+r	Enables read access to all signals for waveform viewing in the GUI
+gui	Opens the SimVision graphical waveform viewer
-coverage all	Enables collection of ALL coverage metrics (line, toggle, branch, expression, FSM)
-covoverwrite	Overwrites any existing coverage database from a previous run

INTERVIEW TIP: In interviews, explain that +access+r grants visibility into internal signals but slows simulation. In production flows, access is limited to improve performance.

4.2 Cadence Genus

Genus is Cadence's logic synthesis tool. It reads RTL code and a technology library, then maps the RTL onto actual standard cells (gates, flip-flops, multiplexers) from the library.

Purpose:

Transform behavioral/structural Verilog RTL into a gate-level netlist of technology-specific standard cells, optimized for area, timing, and power based on SDC constraints.

Inputs:

- RTL source: rtl/ALU.v
- Technology library: lib/slow.lib (Liberty timing library for 90nm slow corner)
- SDC constraints: synthesis_constraints/design_*.sdc

Outputs:

- Synthesized netlist (.v): Gate-level Verilog consisting of standard cell instances
- SDF file (.sdf): Standard Delay Format file with gate and interconnect delays
- Post-synthesis SDC: Updated constraints for physical design
- Reports: timing, area, power, cell count

Key Commands:

Command	Purpose
set_db lib_search_path ./lib	Tell Genus where to find the technology library files
set_db hdl_search_path ./rtl	Tell Genus where to find the Verilog source files
set_db library slow.lib	Load the slow-corner Liberty timing library
read_hdl ALU.v	Parse and read the RTL Verilog file
elaborate ALU	Build the design hierarchy and resolve the top module
read_sdc design_balanced.sdc	Load timing constraints (clock, delays, loads)
syn_generic	First pass: optimize RTL into technology-independent generic gates
syn_map	Second pass: map generic gates to technology-specific standard cells
write_hdl > netlist.v	Export the mapped gate-level netlist
write_sdf > delays.sdf	Export gate delays for back-annotated simulation
write_sdc > post_synth.sdc	Export updated constraints for Innovus
report timing	Show worst setup timing path and slack
report area	Show total cell area
report power	Show leakage, internal, and switching power
report gates	Show cell type breakdown and instance counts

INTERVIEW TIP: Genus uses two synthesis passes: `syn_generic` (technology-independent optimization) and `syn_map` (technology mapping). This is different from Synopsys Design Compiler which uses `compile` or `compile_ultra` as a single command. In an interview, if asked about `compile_ultra`, note that it is a Synopsys DC command, not Genus.

4.3 Cadence Tempus

Tempus is Cadence's sign-off quality static timing analysis (STA) engine. It analyzes all timing paths in a design without running simulation. It checks whether setup and hold constraints are met on every path.

Purpose:

Perform exhaustive timing analysis on synthesized netlists to identify timing violations and verify that the design meets its clock frequency target.

Key Concepts:

Concept	Definition
Setup Check	Ensures data arrives at a flip-flop input BEFORE the clock edge (minus setup time)

Concept	Definition
Hold Check	Ensures data remains stable AFTER the clock edge (plus hold time)
Slack	Difference between required time and arrival time. Positive = MET, Negative = VIOLATED
GBA	Graph-Based Analysis: conservative, fast, used as default
PBA	Path-Based Analysis: more accurate, less pessimistic, uses path-specific slew propagation
Critical Path	The timing path with the worst (smallest or most negative) slack

4.4 Cadence Conformal LEC

Conformal LEC (Logic Equivalence Checking) is a formal verification tool. It mathematically proves that two representations of a design (typically RTL vs. synthesized netlist) are functionally identical.

Key Concepts:

- Golden Design: The reference (original RTL)
- Revised Design: The implementation to verify (synthesized netlist)
- Key Points: Observable points (DFFs, primary outputs) that must match between golden and revised
- Compare Points: Subset of key points selected for verification

4.5 Cadence Innovus

Innovus is Cadence's physical design (Place & Route) tool. It takes a gate-level netlist and produces a physical layout with all cells placed, clocked, and connected by metal wires.

Physical Design Stages in Innovus:

Stage	What Happens	Script Used
Floorplanning	Define die size, core area, I/O pin placement, power ring	floorplan_05.tcl
Power Planning	Create VDD/VSS ring and stripe connections	(inside floorplan_05.tcl)
Placement	Place standard cells in legal row positions	placement.tcl
CTS	Build clock tree with buffers to distribute clock signal	cts.tcl
CTS Optimization	Fix setup/hold violations after CTS	cts_opt.tcl
CTS Repair	Fix DRV violations (transition, capacitance, skew)	cts_repair.tcl
Routing	Connect all signal nets with metal wires	routing.tcl
Signoff	Final DRC/connectivity checks, GDSII export	signoff.tcl

5. RTL Design

5.1 Module Interface

The ALU module has the following port interface:

Port	Direction	Width	Description
clk	Input	1-bit	Clock signal, rising-edge triggered
rst	Input	1-bit	Active-high synchronous reset
in1	Input	8-bit	First operand
in2	Input	8-bit	Second operand
sel	Input	4-bit	Operation select (0000 to 1111)
out1	Output	8-bit	Lower 8-bit result (registered)
out2	Output	8-bit	Upper 8-bit result or overflow (registered)

5.2 Internal Registers

The RTL declares three internal registers that capture the inputs on every rising clock edge:

Register	Width	Purpose
X	8-bit	Registered copy of in1 (captured one cycle before use)
Y	8-bit	Registered copy of in2 (captured one cycle before use)
sel_ff	4-bit	Registered copy of sel (captured one cycle before use)

NOTE: Because X, Y, and sel_ff are updated with non-blocking assignments ($\leq$), the case statement uses the PREVIOUS cycle's values. This means the ALU has a one-clock-cycle pipeline latency. Inputs captured on clock edge N are used for computation on clock edge N+1. This is a very common interview question.

5.3 Operation Encoding

The 4-bit sel input selects one of 16 operations using parameters:

sel Value	Parameter Name	Operation	Result
4'b0000	ADD	Addition	{out2,out1} = X + Y (16-bit result)
4'b0001	SUB	Subtraction	{out2,out1} = X - Y (16-bit result)
4'b0010	MUL	Multiplication	{out2,out1} = X * Y (16-bit product)
4'b0011	DIV	Division	out1 = X / Y, out2 = 0
4'b0100	MOD	Modulus	out1 = X % Y (with div-by-zero guard), out2 = 0
4'b0101	AND	Bitwise AND	out1 = X & Y, out2 = 0
4'b0110	NAND	Bitwise NAND	out1 = ~(X & Y), out2 = 0
4'b0111	OR	Bitwise OR	out1 = X Y, out2 = 0
4'b1000	NOR	Bitwise NOR	out1 = ~(X Y), out2 = 0
4'b1001	XOR	Bitwise XOR	out1 = X ^ Y, out2 = 0
4'b1010	XNOR	Bitwise XNOR	out1 = ~(X ^ Y), out2 = 0

sel Value	Parameter Name	Operation	Result
4'b1011	VEC_REV	Bit Reversal	out1 = reverse(X), out2 = reverse(Y)
4'b1100	PAR	Parity (XOR reduction)	out1 = ^X (1 if odd parity), out2 = 0
4'b1101	SIGN	Sign Detection	out1 = 1 if X[7]=1 (negative), else 0
4'b1110	BIT_EQUAL	Equality Check	out1 = 1 if X==Y, else 0
4'b1111	AVERAGE	Average	out1 = (X + Y) >> 1, out2 = 0

5.4 Synthesizability Analysis

Every construct used in this RTL is synthesizable:

- The single always @(posedge clk) block infers D flip-flops for all registered signals (X, Y, sel_ff, out1, out2).
- The case statement on sel_ff infers a 16:1 multiplexer tree feeding the output registers.
- Arithmetic operators (+, -, *, /, %) infer adders, subtractors, multipliers, and dividers from the standard cell library.
- The division and modulus operations are the most area-expensive, as they require iterative or combinational divider logic (no dedicated hardware divider in the standard cell library).
- The bit reversal operation (VEC_REV) is purely a wiring change and costs zero gates in synthesis. It is just a reordering of bit connections.

WARNING: The division and modulus operators produce the deepest combinational logic paths. The critical path in ALL three synthesized netlists passes through the divider/modulus logic (rem_96_35_Y_div_84_31 instances in the timing reports). This is the performance bottleneck of the design.

5.5 Hardware Generated by Synthesis

The synthesis tool generates the following hardware from this RTL:

- 36 flip-flops total: 8 for X, 8 for Y, 4 for sel_ff, 8 for out1, 8 for out2
- Combinational logic cloud implementing all 16 operations simultaneously
- A 16:1 output MUX tree selecting the correct operation result based on sel_ff
- Adder chains (ADDFX1, ADDFXL full adder cells) for ADD, SUB, MUL, AVERAGE
- Iterative division logic for DIV and MOD (by far the largest and slowest part)

6. Simulation and Testbenches

6.1 Simulation Flow

NCVerilog performs simulation in three phases:

- **Compilation:** The RTL and testbench files are parsed and checked for syntax errors.
- **Elaboration:** The design hierarchy is built, modules are instantiated, and port connections are resolved.
- **Execution:** The simulator runs the initial blocks in the testbench, applies stimuli, toggles the clock, and records waveforms.

6.2 Testbench 1 (tb1.v) - Basic Functional Verification

TB1 is the simplest testbench. It tests all 16 ALU operations with one representative test case each. It uses a 10 ns clock period (5 ns half-period) and applies a reset sequence at the beginning.

Structure: Reset -> Apply input -> Wait 20 ns -> Next operation -> ... -> \$finish

TB1 verifies basic functionality. Each operation is tested once with a single pair of input values.

6.3 Testbench 2 (tb2.v) - Corner Case Testing

TB2 extends TB1 by adding corner-case testing. Key additions:

- **MOD operation:** Tests both dividend=0 and divisor=0 (divide-by-zero handling).
- **Parity:** Tests two different even-parity patterns (11001010 and 10101010).
- **Sign Detection:** Tests both negative (MSB=1) and positive (MSB=0) numbers.
- **Equality:** Tests both equal (0x00==0x00) and unequal (0xFF vs 0x01) operands.
- **Average:** Three test cases including both-equal operands.

6.4 Testbench 3 (tb3.v) - Extended Coverage

TB3 further extends TB2 with:

- **Multiple multiplication test cases:** 7*2, 255*255 (max overflow), 255*0 (multiply-by-zero), and 63*31 (larger operands).
- **Post-verification re-run:** Repeats the ADD operation at the end to verify the ALU continues working correctly after all tests.

TB3 achieves the highest coverage because it exercises the multiplier with diverse operand combinations and re-validates a basic operation at the end.

INTERVIEW TIP: If asked 'Why three testbenches?', answer: TB1 validates basic functionality, TB2 adds corner cases (divide-by-zero, sign boundaries), and TB3 adds multiplier stress tests and a regression re-check. Each builds on the previous to progressively increase coverage.

7. Coverage Analysis

7.1 What is Coverage?

Coverage measures how thoroughly a testbench exercises the design code. It answers the question: 'Did the testbench actually cause every line, every branch, every signal toggle, and every condition to be evaluated?' High coverage gives confidence that the design has been well tested.

7.2 Coverage Types

Coverage Type	What It Measures	Example
Line Coverage	Was every line of RTL code executed at least once?	The line <code>out1 <= X / Y</code> was executed
Toggle Coverage	Did every signal bit toggle from 0->1 and 1->0?	<code>in1[7]</code> toggled both ways
Branch Coverage	Was every branch of if/case taken?	The <code>MOD Y==0</code> branch was entered
Expression Coverage	Were all truth-table combinations of sub-expressions evaluated?	Both <code>(X[7]==1)</code> true and false
FSM Coverage	Were all states and transitions of finite state machines visited?	Not applicable (no FSM in this design)

7.3 How Coverage Was Improved

TB1 alone does not achieve 100% coverage because:

- The MOD divide-by-zero branch (`Y==0`) is never entered in TB1 (TB2 adds this test).
- The sign detection positive-number path is not fully tested (TB2 adds `MSB=0` case).
- The multiplier toggle coverage is incomplete with only one pair of small operands (TB3 adds max/zero/large tests).

By combining all three testbenches and merging their coverage databases, all branches, all signal toggles, and all expression conditions are covered.

INTERVIEW TIP: In an interview, if asked 'How did you reach 100% coverage?', explain the iterative approach: (1) run TB1, (2) analyze the coverage holes in IMC, (3) write TB2 to hit the missed branches, (4) re-analyze, (5) write TB3 for remaining toggle gaps. This iterative coverage-driven approach is standard industry practice.

8. Constraints (SDC)

SDC (Synopsys Design Constraints) is a standard TCL-based format used by all major synthesis and STA tools. It defines the timing environment: clock definitions, input/output delays, driving cells, loads, and timing exceptions.

Three SDC files were used in this project, one for each synthesis strategy:

8.1 Constraint Comparison

Constraint	Min Area SDC	Balanced SDC	Tight Timing SDC
Clock Period	15 ns (66.67 MHz)	8 ns (125 MHz)	4 ns (250 MHz)
Clock Transition (rise/fall)	0.4 ns	0.4 ns	0.4 ns
Clock Uncertainty (setup)	0.4 ns	0.4 ns	0.4 ns
Input Transition (in1, in2, sel)	0.2 ns	0.2 ns	0.2 ns
Output Load (out1, out2)	1 unit	1 unit	1 unit
Input Delay	Not specified	Not specified	Not specified
Output Delay	Not specified	Not specified	Not specified
False Path	None	None	None
Multicycle Path	None	None	None

8.2 Detailed Constraint Explanation

8.2.1 create_clock

```
create_clock -name clk -period 8 [get_ports clk]
```

WHAT: Creates a clock definition named 'clk' with a specified period, attached to the port named 'clk'.

WHY: Every synchronous design needs at least one clock. The clock period determines the maximum allowed combinational delay between any two flip-flops.

HOW: The STA tool uses this to calculate Required Time = clock period - setup time - uncertainty.

Impact: A shorter clock period forces the synthesis tool to use faster (larger, higher-power) cells to meet timing. A longer period allows the tool to use smaller, slower cells, saving area and power.

8.2.2 set_clock_transition

```
set_clock_transition -rise 0.4 [get_clocks clk]
set_clock_transition -fall 0.4 [get_clocks clk]
```

WHAT: Sets the rise and fall transition time (slew rate) of the clock signal to 0.4 ns.

WHY: A real clock signal does not transition instantaneously. The transition time models how fast the clock edge rises from 0 to VDD or falls from VDD to 0.

Impact: A slower clock transition increases the setup and hold times of flip-flops, consuming timing margin. The 0.4 ns value is a moderately realistic assumption for a 90nm process.

8.2.3 set_clock_uncertainty

```
set_clock_uncertainty -setup 0.4 [get_clocks clk]
```

WHAT: Adds 0.4 ns of timing uncertainty (also called timing margin or clock pessimism) to setup analysis.

WHY: In real silicon, clock jitter, manufacturing variation, and on-chip variation (OCV) cause the clock to arrive at different flip-flops at slightly different times. Clock uncertainty reserves margin for these unpredictable effects.

Impact: The 0.4 ns uncertainty directly subtracts from the available timing budget. Effective clock period for timing = period - uncertainty = 8.0 - 0.4 = 7.6 ns.

INTERVIEW TIP: Clock uncertainty is one of the most commonly asked concepts in interviews. The formula is: Available Timing Budget = Clock Period - Setup Time - Clock Uncertainty. A higher uncertainty means less budget for combinational logic delay.

8.2.4 set_input_transition

```
set_input_transition 0.2 [get_ports in1]
```

WHAT: Models the transition time (slew) of signals arriving at input ports from external logic.

WHY: The first gate in the design sees a signal with a finite transition time from the upstream block. A slower input transition increases the delay of the first cell.

8.2.5 set_load

```
set_load 1 [get_ports out1]
```

WHAT: Places a capacitive load of 1 unit on the output ports.

WHY: In a real chip, output ports drive downstream logic or pads which present a capacitive load. This load increases the delay of the last gate driving the output.

Impact: Higher load = higher delay for the last stage = more timing pressure. A load of 1 pF is a moderate external load for 90nm.

8.3 Constraints Not Used (But Important to Know)

Constraint	What It Does	Why Not Used Here
set_input_delay	Models delay of signals arriving at inputs relative to clock	Simplification: inputs assumed synchronous with no external delay
set_output_delay	Models required arrival time at outputs relative to clock	Simplification: no downstream timing requirement specified
set_driving_cell	Models the drive strength of external logic feeding inputs	set_input_transition used as simpler alternative
set_max_fanout	Limits maximum number of gates a single output can drive	Default library limits used
set_false_path	Excludes a path from timing analysis	No asynchronous crossings or test-only paths
set_multicycle_path	Allows a path more than one clock cycle to complete	All paths are single-cycle by design

9. Synthesis

9.1 Synthesis Flow Overview

Genus synthesis converts RTL into a gate-level netlist in two stages:

- `syn_generic`: Translates RTL into technology-independent generic logic (generic AND, OR, MUX, adder). Performs architecture-level optimizations.
- `syn_map`: Maps generic logic onto specific standard cells from the technology library (`slow.lib`). Performs cell-level optimizations like gate sizing and logic restructuring.

9.2 Why Three Netlists?

Three synthesis runs were performed with identical RTL but different clock periods in the SDC constraints. This demonstrates the fundamental area-timing-power trade-off in ASIC design:

- **Min Area** (15 ns, 66.67 MHz): Very relaxed timing gives the synthesis tool freedom to use the smallest, slowest cells. Result: smallest area, lowest power, slowest speed.
- **Balanced** (8 ns, 125 MHz): Moderate timing pressure forces moderate-speed cells. Result: middle ground in area, power, and speed.
- **Tight Timing** (4 ns, 250 MHz): Aggressive timing pressure forces the tool to use the largest, fastest cells with maximum drive strength. Result: largest area, highest power, fastest speed.

9.3 Synthesis Results Comparison

Metric	Min Area	Balanced	Tight Timing
Clock Period	15 ns	8 ns	4 ns
Frequency	66.67 MHz	125 MHz	250 MHz
Cell Count	450	636	1146
Total Cell Area	3745.898 um2	4198.524 um2	6324.656 um2
Sequential Area	718.298 (19.2%)	736.464 (17.5%)	744.790 (11.8%)
Combinational Area	2975.374 (79.4%)	3296.299 (78.5%)	4946.341 (78.2%)
Inverter Area	52.226 (1.4%)	161.220 (3.8%)	423.107 (6.7%)
Buffer Count	0	1	30
Total Power	244.78 uW	513.24 uW	1142.36 uW
Leakage Power	19.25 uW (7.86%)	20.56 uW (4.01%)	32.73 uW (2.87%)
Internal Power	100.07 uW (40.88%)	214.29 uW (41.75%)	560.48 uW (49.06%)
Switching Power	125.45 uW (51.25%)	278.39 uW (54.24%)	549.15 uW (48.07%)
Setup Slack	+1825 ps (MET)	+549 ps (MET)	+40 ps (MET)
Critical Endpoint	out1_reg[5]/D	out1_reg[2]/D	out1_reg[0]/D
Critical Startpoint	Y_reg[7]	Y_reg[5]	Y_reg[0]

9.4 Analysis of Results

Key observations from the synthesis comparison:

Area Scaling:

The tight timing netlist (6324.656 um2) is 1.69x the area of the min area netlist (3745.898 um2). This is because the synthesis tool uses higher-drive-strength variants of the same cells (e.g., DFFHQX8 instead of DFFQX1, NAND2X4 instead of NAND2XL) and adds buffers (30 buffers vs. 0) to reduce transition times on critical paths.

Power Scaling:

Total power increases by 4.67x from min area (244.78 μ W) to tight timing (1142.36 μ W). This is because larger cells have higher internal capacitance (more internal power) and the higher frequency doubles the switching activity.

Sequential Area is Nearly Constant:

All three netlists have exactly 36 flip-flops. The sequential area only changes slightly (718-745 μ m²) because the tool selects higher-drive-strength flip-flop variants for tight timing (DFFHQX8 vs DFFQX1).

Critical Path:

In all three netlists, the critical path passes through the division/modulus combinational logic (rem_96_35_Y_div_84_31 hierarchy). This is because integer division requires iterative subtraction logic, which produces the deepest logic depth.

INTERVIEW TIP: If asked 'Why does the area increase when timing gets tighter?', answer: The synthesis tool upsizes cells to higher drive strengths (NAND2XL -> NAND2X2 -> NAND2X4) and inserts buffers to reduce wire delay and transition times. Higher drive strength means physically larger transistors, hence more area.

10. Logic Equivalence Checking (LEC)

10.1 What is LEC?

Logic Equivalence Checking is a formal verification method that mathematically proves two designs are functionally identical. Unlike simulation (which can only test a finite number of input patterns), LEC exhaustively proves equivalence for ALL possible input combinations.

10.2 LEC Flow

- Read the technology library (lib/slow.v - Verilog models of standard cells)
- Read the Golden design (original RTL: rtl/ALU.v)
- Read the Revised design (synthesized netlist)
- Enter LEC mode (set system mode lec)
- Map key points between golden and revised (add compared point -all)
- Run comparison (compare)
- Generate reports

10.3 LEC Results - Good Netlists

Metric	Min Area	Balanced	Tight Timing
Golden PI (Primary Inputs)	22	22	22
Golden PO (Primary Outputs)	16	16	16
Golden DFF	36	36	36
Total Mapped Points	74	74	74
Compared Points	52	52	52
Equivalent Points	52	52	52
Non-Equivalent Points	0	0	0
Result	PASS	PASS	PASS

NOTE: 52 compared points = 16 POs + 36 DFFs. The 22 PIs are not compared because they are sources, not destinations. Equivalence is checked at observable output points.

10.4 LEC Results - Bad Netlists

Three intentionally corrupted netlists were created to demonstrate LEC's defect detection capability:

Bad Netlist	Bug Injected	Source Base	Result	Non-Equiv Points
bad1_missing_ff.v	Deleted Y_reg[7] flip-flop instance	Min Area	FAIL:NONEQ	16 DFFs
bad2_miswire.v	Changed NAND3BX1 input net n_58 to n_57	Tight Timing	FAIL:NONEQ	8 DFFs
bad3_wrong_cell.v	Replaced OAI21X1 with OAI22X1	Balanced	FAIL:NONEQ	1 DFF

10.4.1 Bad1 Analysis: Missing Flip-Flop

When Y_reg[7] is deleted, the signal Y[7] becomes undriven. This cascades to every output that depends on Y[7], which includes all out1 and out2 registers (because the division, modulus, and arithmetic operations all use the full 8-bit Y). Result: 16 out of 36 DFFs are non-equivalent, and 1 DFF is unmapped (Y_reg[7] exists in golden but not in revised).

10.4.2 Bad2 Analysis: Miswired Net

Changing one input pin of a NAND gate from n_58 to n_57 corrupts the Boolean function of the logic cone feeding the out2 register bank. Since this net is in the upper-bit path of the multiplication/division result, all 8 out2 DFFs become non-equivalent. The out1 and X/Y/sel_ff DFFs are unaffected because their logic cones do not include this specific gate.

10.4.3 Bad3 Analysis: Wrong Cell Type

Replacing OAI21X1 (OR-AND-INVERT 2-1) with OAI22X1 (OR-AND-INVERT 2-2) changes the Boolean function of one gate. The B0 input in OAI21X1 is a single-input OR, but in OAI22X1 it becomes part of a 2-input OR group. This affects exactly 1 DFF: out1_reg[0]. This is the most precise defect detection - LEC pinpoints exactly which register is affected.

INTERVIEW TIP: In an interview, LEC questions frequently ask about the difference between LEC and simulation. Key answer: Simulation checks a FINITE number of input vectors. LEC is EXHAUSTIVE - it mathematically proves equivalence for ALL possible input combinations using formal Boolean methods. Simulation can miss corner-case bugs that LEC will always catch.

11. Static Timing Analysis (STA)

11.1 STA Fundamentals

Static Timing Analysis checks all timing paths in a design without running simulation. It computes the delay through every combinational path between flip-flops and verifies that data arrives before the clock edge (setup check) and stays stable after the clock edge (hold check).

11.1.1 Key Timing Formulas

Formula	Equation
Setup Slack	Slack = Required Time - Arrival Time
Required Time (Setup)	Clock Period - Setup Time - Clock Uncertainty
Arrival Time	Clock-to-Q delay of launch flop + combinational path delay
Hold Slack	Arrival Time - (Hold Time + Clock Uncertainty_hold)

Positive Slack = TIMING MET (good). Negative Slack = TIMING VIOLATED (bad).

11.2 STA Strategy in This Project

All three synthesized netlists were analyzed in Tempus using the SAME 8 ns balanced constraint (design_balanced.sdc). This creates a fair comparison because the timing target is identical; only the netlist quality differs.

11.3 STA Results Comparison (GBA Setup Analysis at 8 ns Period)

Metric	Min Area Netlist	Balanced Netlist	Tight Timing Netlist
Setup Slack	-5.220 ns (VIOLATED)	+0.506 ns (MET)	+4.021 ns (MET)
Data Path Delay	12.809 ns	6.986 ns	3.471 ns
Required Time	7.588 ns	7.492 ns	7.492 ns
Clock Period	8.000 ns	8.000 ns	8.000 ns
Setup Time	0.012 ns	0.108 ns	0.108 ns
Critical Endpoint	out1_reg[2]	out1_reg[2]	out1_reg[0]
Result	VIOLATED	MET	MET

11.4 Why Min Area Netlist Fails at 8 ns

The min area netlist was synthesized with a 15 ns clock period. When analyzed at 8 ns, it VIOLATES timing with -5.220 ns slack because:

- The synthesis tool used the smallest, slowest cells (DFFQX1 instead of DFFQX4, NAND2XL instead of NAND2X1).
- No buffers were inserted to reduce high-fanout wire delay.
- The data path delay is 12.809 ns, far exceeding the 7.588 ns required time.

This demonstrates that a netlist optimized for one frequency cannot simply run at a higher frequency. The cells are too slow.

11.5 Why Tight Timing Has the Most Slack at 8 ns

The tight timing netlist was synthesized with a 4 ns clock period. When analyzed at 8 ns (twice its target), it has +4.021 ns of positive slack because the cells are massively oversized. It was designed for 250 MHz speed, so running at 125 MHz is trivially easy.

11.6 Hold Timing Analysis

All three netlists show HOLD VIOLATIONS in pre-CTS STA. This is expected and normal:

Netlist	Worst Hold Slack	Endpoint	Reason
Min Area	-0.089 ns	X_reg[0]	Input arrives instantly (no input delay); too fast for hold
Balanced	-0.134 ns	Y_reg[0]	Same reason
Tight Timing	-0.157 ns	X_reg[1]	Higher-drive flip-flops have larger hold time requirements

NOTE: Hold violations on input-to-register paths are expected in pre-CTS STA because no input delay was specified in the SDC. In real designs, external logic always has some delay. These hold violations are fixed later in the physical design flow by inserting hold-fixing buffers during CTS optimization.

INTERVIEW TIP: If asked 'Why do you have hold violations?', answer: These are input-to-first-register paths with zero input delay specified. In a real SoC, the upstream block provides some delay. These are fixed during CTS hold optimization by inserting delay buffers. Hold violations are NEVER fixed by changing the clock period (hold is independent of clock period).

11.7 STA Parametric Experiments

Additional experiments were performed on the min area netlist with the balanced 8 ns constraint:

Experiment	Parameter Changed	Setup Slack
Baseline	Default (slew 0.4, load 1)	-5.220 ns
Ideal Clock Slew	Clock transition = 0.0 ns	-5.215 ns
Slow Clock Slew	Clock transition = 0.8 ns	-5.218 ns
Zero Output Load	Output load = 0.0 pF	-5.220 ns
Double Output Load	Output load = 2.0 pF	-5.220 ns

Key observation: The setup slack barely changes across these experiments. This is because the critical path is internal (register-to-register through the divider logic), not input-to-output. The output load and input slew only affect paths that start at primary inputs or end at primary outputs, which are not the critical path.

12. Design for Testability (DFT)

12.1 Why DFT is Required

After a chip is manufactured, it must be tested to ensure no manufacturing defects (e.g., stuck-at-0, stuck-at-1 faults, bridging faults) exist. DFT modifies the design to make it testable by Automatic Test Pattern Generation (ATPG) tools.

Without DFT, testing internal flip-flops requires controlling and observing them through primary I/O, which is extremely difficult for complex designs. DFT solves this by converting flip-flops into scan flip-flops that can be chained together like a shift register.

12.2 Scan Chain Architecture

In this project, all 36 flip-flops were converted to scan flip-flops and connected into a single scan chain:

Parameter	Value
Scan Style	Muxed Scan (multiplexer-based)
Number of Scan Chains	1 (AutoChain_1)
Chain Length	36 flip-flops
Scan Input Port	DFT_sdi_1 (newly created)
Scan Output Port	DFT_sdo_1 (newly created)
Shift Enable Port	scan_en (active high, newly created)
Clock Domain	clk (rising edge)

12.3 Scan Chain Order

The scan chain connects flip-flops in the following order (bit 1 through bit 36):

X_reg[0:7] -> Y_reg[0:7] -> out1_reg[0:7] -> out2_reg[0:7] -> sel_ff_reg[0:3]

12.4 How Muxed Scan Works

Each regular D flip-flop (DFFQX1) is replaced by a scan D flip-flop (SDFFQX1 or SDFFHQX8). The scan flip-flop has an additional multiplexer at its input:

- When scan_en = 0 (Normal Mode): The flip-flop captures data from its functional D input (normal operation).
- When scan_en = 1 (Shift Mode): The flip-flop captures data from its SI (Scan Input) port, which is connected to the Q output of the previous flip-flop in the chain.

12.5 DFT Impact on Area and Timing

Metric	Pre-DFT (Balanced)	Post-DFT (Balanced)	Change
Cell Count	636	623	-13 (net reduction from cell merging)
Total Area	4198.524 um2	4331.739 um2	+133.215 um2 (+3.2%)
Sequential Area	736.464 um2	905.252 um2	+168.788 um2 (+22.9%)
Combinational Area	3296.299 um2	3283.432 um2	-12.867 um2 (-0.4%)
Setup Slack (at 8 ns)	+549 ps	+208 ps	-341 ps (degraded)
Total Power	513.24 uW	520.75 uW	+7.51 uW (+1.5%)

The area increases because scan flip-flops are larger than regular flip-flops (they include a 2:1 mux). The timing slack degrades because the scan mux adds delay to the data input path of every flip-flop.

INTERVIEW TIP: If asked 'Why does area increase after scan insertion?', answer: Every D flip-flop is replaced with a scan flip-flop that includes a 2:1 multiplexer for selecting between functional data (D) and scan data (SI). This mux adds both area and delay. The area increase is typically 3-15% depending on the design.

13. Post-DFT Static Timing Analysis

13.1 Why Post-DFT STA is Required

After scan insertion, timing must be re-verified because:

- Scan muxes add delay to the D-input path of every flip-flop, increasing data arrival time.
- Flip-flop types may change (e.g., DFFQX1 -> SDFFQX1), which changes setup/hold times.
- New ports (scan_en, DFT_sdi_1, DFT_sdo_1) create new timing paths.

13.2 Post-DFT STA Configuration

The post-DFT STA script sets scan_en = 0 using set_case_analysis to analyze only functional timing paths (not scan shift paths):

```
set_case_analysis 0 [get_ports scan_en]
```

This ensures the STA only checks normal operating-mode timing, not the scan shift timing.

13.3 Post-DFT Timing Results

Metric	Pre-DFT STA	Post-DFT STA	Change
Setup Slack (GBA)	+0.506 ns	+0.163 ns	-0.343 ns (degraded)
Hold Slack (worst)	-0.134 ns	-0.063 ns	+0.071 ns (improved)
Critical Endpoint	out1_reg[2]	out1_reg[2]	Same
Critical Startpoint	Y_reg[5]	Y_reg[1]	Changed
Setup Time of Endpoint	0.108 ns	0.185 ns	+0.077 ns (scan FF is slower)

Setup slack degraded by 0.343 ns because the scan mux adds delay. Hold slack improved slightly because the scan flip-flop type has a smaller hold time requirement. The critical startpoint changed because the scan mux redistributes the timing path delays.

14. Physical Design

Physical design is the process of converting a gate-level netlist into a physical layout with cells placed on silicon and connected by metal wires. This is the largest and most complex stage of the ASIC flow.

14.1 MMMC Setup

Innovus uses Multi-Mode Multi-Corner (MMMC) analysis. For this project, a single analysis view was configured:

Component	Setting
Library Set	timing_lib1: lib/slow.lib
Constraint Mode	sdcl: dft_reports/balanced/dft_constraints_balanced.sdc
Delay Corner	delay1 (early = slow, late = slow, single corner)
Analysis View	view1 (setup = view1, hold = view1)

14.2 Floorplanning

14.2.1 Die Size Calculation

The floorplan was calculated mathematically:

Parameter	Value	How Calculated
Post-DFT Cell Area	4331.739 um ²	From DFT area report
Target Utilization	50% (0.5)	Project specification
Required Core Area	8663.478 um ²	4331.739 / 0.5
Ideal Square Side	93.078 um	sqrt(8663.478)
Library Row Height	3.36 um	gsclib090site row height
Snapped Row Count	28 rows	ceil(93.078 / 3.36)
Actual Core Side	94.08 um	28 * 3.36
Core-to-Die Margin	4.06 um each side	For power ring placement
Final Die Dimensions	~103.24 x 99.47 um	From design boundary reports

```
floorPlan -site gsclib090site -r 1 0.5 4.06 4.06 4.06 4.06
```

The -r 1 sets aspect ratio to 1:1 (square). The 0.5 sets utilization to 50%. The four 4.06 values are left/bottom/right/top margins for the power ring.

14.2.2 I/O Pin Placement

Pins were placed on all four sides of the die:

Side	Pins Placed	Signal Groups
North	clk, rst, scan_en, DFT_sdi_1, DFT_sdo_1	Control and DFT signals
West	in1[0:7]	First operand bus
South	in2[0:7], sel[0:3]	Second operand + select bus
East	out1[0:7], out2[0:7]	Output buses

14.2.3 Power Ring

A VDD/VSS power ring was created around the core:

Parameter	Value
Ring Width	1.25 um
Ring Spacing	0.4 um
Horizontal Layers	Metal8 (top and bottom)
Vertical Layers	Metal9 (left and right)
Via Stack	Metal1 through Metal9
Nets	VDD and VSS

14.3 Placement

Standard cell placement was performed using Innovus's timing-driven placement engine. Scan chains were defined before placement to ensure scan-connected flip-flops are placed near each other:

```
specifyScanChain scan1 -start DFT_sdi_1 -stop DFT_sdo_1
specifyScanChain scan2 -start DFT_sdi_2 -stop DFT_sdo_2
setPlaceMode -fp false
placeDesign
```

14.3.1 Post-Placement Timing

After placement (before CTS), timing was VIOLATED:

Metric	Post-Placement Value
Worst Setup Slack	-2.257 ns (VIOLATED)
Critical Path Endpoint	out1_reg[6]/D
Hold Slack (worst)	-0.228 ns (VIOLATED)
DRC Violations	995
Connectivity Violations	0

NOTE: Setup violations after placement are expected because: (1) the clock is still ideal (no clock tree built yet), (2) real wire parasitics now add delay that was not present in synthesis. The CTS and optimization steps will fix these.

14.4 Clock Tree Synthesis (CTS)

CTS builds a physical clock distribution network using clock buffers and inverters to deliver the clock signal to all 36 scan flip-flops with balanced insertion delay and minimal skew.

14.4.1 CTS Buffer Cells

The following clock buffer cells were specified for CTS:

CLKBUF3, CLKBUF2, CLKBUF8, CLKBUF12, CLKBUF16, CLKBUF20, CLKINV3, CLKINV2, CLKINV8, CLKINV12, CLKINV16

The ccopt_design command builds the optimal clock tree automatically. After CTS, the clock network latency changes from ideal (0) to propagated (actual buffer delays). This typically IMPROVES setup timing because launch and capture clocks now have balanced delays.

14.4.2 Post-CTS Timing

Stage	Setup Slack	Status
Post-Placement (ideal clock)	-2.257 ns	VIOLATED
Post-CTS	+0.041 ns	MET
Post-CTS Optimization	+0.037 ns	MET
Post-CTS Repair	+0.047 ns	MET

The dramatic improvement from -2.257 ns to +0.041 ns is because the CTS inserts buffers and balances the clock tree. With propagated clock, both launch and capture paths have similar delays, so the relative timing budget is restored. The CTS optimization (optDesign -postCTS and optDesign -postCTS -hold) further fine-tunes timing.

14.5 Routing

After CTS repair, all signal nets were routed using Innovus's NanoRoute engine:

```
routeDesign -globalDetail
```

NanoRoute performs both global routing (coarse path planning) and detailed routing (exact metal track assignment) in a single step with -globalDetail.

14.5.1 Post-Route Timing

Metric	Post-Route Value
Worst Setup Slack	+0.059 ns (MET)
Critical Endpoint	out1_reg[3]/D
Worst Hold Slack	+0.010 ns (MET)
Hold Path	scan_en -> out1_reg[0]/SE

Both setup and hold timing are clean after routing. The design achieves timing closure at 125 MHz.

14.5.2 Post-Route Area

Metric	Post-Placement	Post-Route
Cell Count	623	661
Total Area	4331.739 um2	4567.891 um2
Buffer Area	0.000 um2	326.981 um2
Sequential Area	905.252 um2	762.955 um2
Combinational Area	3283.432 um2	3309.924 um2

The cell count increased from 623 to 661 (+38 cells) due to buffer insertion during CTS and timing optimization. Buffer area grew from 0 to 326.981 um2. Some scan flip-flops were downsized (905 -> 763 um2) as the optimizer traded sequential drive strength for additional buffers.

14.6 Signoff

14.6.1 DRC Results

Stage	DRC Violations	Breakdown
Post-Placement	995	884 Metal2 margin + 99 outOfDie + 12 Via2 adjCut
Post-Route (Signoff)	12	8 Metal1 outOfDie + 4 Metal1 short

The 12 remaining DRC violations are edge effects from the power ring at the die boundary (outOfDie) and minor Metal1 shorts near power rail connections. In an industry flow, these would be resolved with a DRC-clean power grid redesign or waived if they are within foundry-accepted tolerances.

14.6.2 Connectivity Results

Connectivity verification passed with 0 violations and 0 warnings. All nets are properly connected.

14.6.3 Final Outputs

Output	File	Purpose
GDSII Layout	alu_final_05.gds	Physical layout for fabrication mask generation
Post-Route Netlist	alu_post_route_netlist.v	Final gate-level netlist with physical optimizations
DEF File	alu_final.def	Physical placement, floorplan, and routing geometry

15. Complete Timing Progression Summary

This table tracks the setup slack through every stage of the flow:

Stage	Tool	Setup Slack	Status
Synthesis (Balanced, 8 ns)	Genus	+549 ps	MET
STA (Balanced netlist, 8 ns)	Tempus	+506 ps	MET
Post-DFT STA (8 ns)	Tempus	+163 ps	MET
DFT Genus Internal	Genus	+208 ps	MET
Post-Placement (ideal clk)	Innovus	-2257 ps	VIOLATED
Post-CTS	Innovus	+41 ps	MET
Post-CTS Optimization	Innovus	+37 ps	MET
Post-CTS Repair	Innovus	+47 ps	MET
Post-Route (Final)	Innovus	+59 ps	MET

INTERVIEW TIP: Memorize this progression. Interviewers love asking 'Walk me through how timing changed at each stage.' The key insight: slack **DEGRADED** after placement due to real parasitics, then **RECOVERED** after CTS because the clock tree balanced launch and capture delays.

16. Final Design Summary

Parameter	Final Value
Technology Node	90 nm
Target Frequency	125 MHz (8 ns period)
Final Setup Slack	+59 ps (MET)
Final Hold Slack	+10 ps (MET)
Total Cell Area	4567.891 um ²
Die Size	~103.24 um x 99.47 um
Utilization	~50%
Cell Count (Final)	661
Flip-Flop Count	36 (all scan-converted)
Scan Chains	1 chain, 36 bits
Power Ring Layers	Metal8 (H) / Metal9 (V)
DRC Violations	12 (power ring edge effects)
Connectivity Violations	0
LEC Status	PASS (all three netlists)
GDSII	Streamed out successfully

17. Complete Interview Preparation

This chapter contains 120+ interview questions with answers, all specific to this project. Study this chapter thoroughly.

Q1: What does the ALU do?

The ALU performs 16 arithmetic and logical operations on two 8-bit inputs, selected by a 4-bit control signal. It produces two 8-bit outputs. It is fully synchronous with a single clock and has one-cycle pipeline latency.

Q2: Why does the ALU have a one-cycle latency?

Because the inputs (in1, in2, sel) are registered into internal flip-flops (X, Y, sel_ff) using non-blocking assignments. The case statement operates on the PREVIOUS cycle's captured values, so the result appears one cycle after the input changes.

Q3: What is the critical path and why?

The critical path passes through the division/modulus combinational logic (rem_96_35_Y_div_84_31 hierarchy). Division requires iterative subtraction logic which produces the deepest logic depth. This is true for all three netlists.

Q4: How many flip-flops are in the design?

36 total: 8 (X) + 8 (Y) + 4 (sel_ff) + 8 (out1) + 8 (out2). This count is constant across all three netlists.

Q5: What happens if Y is zero during MOD operation?

The RTL explicitly checks for $Y=0$ and sets out1=0, out2=0. This prevents a divide-by-zero error. For DIV, this guard is NOT present in the RTL, so dividing by zero would produce an unknown result.

Q6: Which operation costs zero gates?

VEC_REV (bit reversal). It is purely a wiring reorder: out1 connects X[0] to bit 7, X[1] to bit 6, etc. No logic gates are needed.

Q7: Why three synthesized netlists?

To demonstrate the area-timing-power trade-off. Min Area uses a relaxed 15ns clock (smallest cells, lowest power, slowest). Balanced uses 8ns (moderate). Tight Timing uses 4ns (largest cells, highest power, fastest). Same RTL, different constraints.

Q8: What does syn_generic do?

syn_generic converts RTL into technology-independent generic gates (AND, OR, MUX, adder). It performs architecture-level optimizations like constant propagation, dead code removal, and resource sharing.

Q9: What does syn_map do?

syn_map maps the generic gates onto specific standard cells from the technology library (slow.lib). It performs gate sizing, logic restructuring, and timing-driven optimization.

Q10: Why is the tight timing netlist 1.69x larger?

Because Genus uses higher-drive-strength cell variants (NAND2X4 vs NAND2XL), adds 30 buffers for wire delay reduction, and uses faster flip-flops (DFFHQX8 vs DFFQX1). Larger transistors = more area.

Q11: What is the difference between compile and compile_ultra?

These are Synopsys Design Compiler commands, NOT Genus commands. compile does basic synthesis. compile_ultra enables advanced optimizations like datapath optimization and boundary optimization. In Genus, the equivalent is syn_generic + syn_map, and syn_opt for further optimization.

Q12: Why is the sequential area nearly constant across netlists?

Because all three netlists have exactly 36 flip-flops. The small variation (718-745 um²) comes from the tool selecting different drive-strength variants of the same flip-flop type.

Q13: What is clock uncertainty and why is it 0.4 ns?

Clock uncertainty models jitter, OCV, and skew that cannot be predicted at synthesis time. 0.4 ns is about 5% of the 8ns period, a reasonable margin for 90nm. It directly subtracts from the available timing budget.

Q14: What happens if you reduce the clock period?

The synthesis tool must use faster, larger cells to close timing. Area increases, power increases. If the period is too small, timing cannot be closed (negative slack).

Q15: Why no input_delay or output_delay?

This is a simplification. In a real SoC, these would be set based on the timing of adjacent blocks. Their absence means inputs are assumed synchronous with zero delay and outputs have no downstream timing requirement.

Q16: What is the effective timing budget?

For the balanced case: 8.0 ns (period) - 0.4 ns (uncertainty) - 0.108 ns (setup time) = 7.492 ns. This is the Required Time shown in the timing reports.

Q17: What is clock transition and why does it matter?

Clock transition is the time for the clock to rise from 0 to VDD (or fall). Set to 0.4 ns in all three SDCs. A slower transition increases flip-flop setup/hold times and adds delay to the clock path.

Q18: What is the difference between golden and revised?

Golden is the reference design (original RTL). Revised is the implementation to verify (synthesized netlist). LEC proves they are functionally identical.

Q19: Why 52 compared points, not 74 mapped points?

74 mapped points = 22 PI + 16 PO + 36 DFF. Only POs and DFFs are compared (52 = 16 + 36). PIs are sources (inputs), so they don't need equivalence checking.

Q20: Why did bad1 (missing FF) cause 16 non-equivalent points?

Deleting Y_reg[7] makes Y[7] undriven. All 16 output registers (out1[0:7] and out2[0:7]) depend on Y[7] through arithmetic operations, so all 16 become non-equivalent.

Q21: Why did bad3 (wrong cell) only cause 1 non-equivalent point?

The cell swap (OAI21X1 to OAI22X1) affects only one specific logic cone. Only out1_reg[0] is in the fanout of that gate, so only 1 DFF fails.

Q22: What is the difference between simulation and LEC?

Simulation tests a FINITE number of input patterns. LEC is EXHAUSTIVE - it uses formal Boolean methods to prove equivalence for ALL possible inputs. LEC catches bugs that simulation might miss.

Q23: What is setup slack?

Setup Slack = Required Time - Arrival Time. Positive = timing MET. Negative = timing VIOLATED. It measures how much timing margin exists before the data would arrive too late.

Q24: What is hold slack?

Hold slack measures whether data remains stable long enough after the clock edge. Hold Slack = Arrival Time - Hold Time. Negative = data changes too quickly.

Q25: Why does the min area netlist fail at 8ns?

Because it was synthesized with a 15ns constraint. The smallest, slowest cells were selected. The data path delay is 12.809 ns, exceeding the 7.588 ns required time by 5.220 ns.

Q26: What is the difference between GBA and PBA?

GBA (Graph-Based Analysis) is conservative - it computes worst-case slew at each node independently. PBA (Path-Based Analysis) propagates actual slew along a specific path, giving less pessimistic (more accurate) results.

Q27: What is the difference between Tempus and PrimeTime?

Tempus is Cadence's STA tool. PrimeTime is Synopsys's STA tool. Both perform the same function (static timing analysis). Tempus integrates natively with Innovus (Cadence PnR). PrimeTime integrates with ICC2 (Synopsys PnR).

Q28: What is a scan chain?

A scan chain connects all flip-flops in a serial shift-register configuration for manufacturing test. In shift mode (scan_en=1), test patterns are shifted in through DFT_sdi_1 and results shifted out through DFT_sdo_1.

Q29: Why did DFT increase area by 3.2%?

Because each DFFQX1 was replaced with SDDFFQX1 or SDDFFHQX8 which includes a 2:1 scan mux. The mux requires additional transistors, increasing area from 4198.524 to 4331.739 um².

Q30: Why did setup slack degrade after DFT?

The scan mux adds delay to the D-input path. The scan FF setup time also increased (0.108 -> 0.185 ns). Combined, this reduced slack from +506 ps to +163 ps.

Q31: What is ATPG?

Automatic Test Pattern Generation. It generates input patterns that detect manufacturing faults (stuck-at-0, stuck-at-1) by controlling scan chain inputs and observing outputs. The project exports ATPG patterns in the file atpg_balanced.atpg.

Q32: What is fault coverage?

Fault coverage = (detected faults / total possible faults) x 100%. Industry targets are typically >95%. DFT scan insertion enables high fault coverage by making internal flip-flops controllable and observable.

Q33: Why 50% utilization?

50% utilization leaves half the core area free for routing channels, clock buffers, and optimization buffers. Higher utilization (>70%) causes routing congestion and makes timing closure difficult. Lower utilization wastes silicon area.

Q34: How was the die size calculated?

Cell Area (4331.739) / Utilization (0.5) = Required Area (8663.478 μm^2). Square root = 93.078 μm . Snapped to 28 library rows (28 * 3.36 = 94.08 μm) + margins (4.06 μm each side).

Q35: Why Metal8/Metal9 for power ring?

Higher metal layers are thicker and have lower resistance, making them ideal for carrying high-current power signals. Metal1-Metal7 are reserved for signal routing.

Q36: Why was timing violated after placement?

Two reasons: (1) Real wire parasitics (resistance and capacitance) add delay that was not present in synthesis (which used zero-wireload). (2) The clock was still ideal (zero latency).

Q37: How did CTS fix the -2.257 ns violation?

CTS built a real clock tree with buffers. With propagated clock, both launch and capture paths have balanced delays. The launch clock delay cancels most of the capture clock delay, restoring the relative timing budget. Additionally, CTS optimization resized cells on critical paths.

Q38: What does optDesign -postCTS do?

It optimizes setup timing after CTS by resizing cells, adding buffers, and restructuring logic on violated paths.

Q39: What does optDesign -postCTS -hold do?

It fixes hold violations by inserting delay buffers on short paths to slow down data arrival and meet hold time requirements.

Q40: What is ccopt_design?

ccopt_design is the Clock Concurrent Optimization command that builds the clock tree while simultaneously optimizing timing. It replaces the older clockDesign command.

Q41: What does routeDesign -globalDetail do?

It performs both global routing (coarse path planning across the chip) and detailed routing (assigning exact metal tracks and vias) in one step.

Q42: Why are there 12 DRC violations at signoff?

8 outOfDie violations (Metal1 shapes extending slightly beyond die boundary from power ring) and 4 Metal1 shorts (adjacent power rail connections). These are power grid edge effects, not signal routing problems.

Q43: What is a LEF file?

Library Exchange Format. Contains physical information about standard cells: cell dimensions, pin locations, metal layer shapes, and obstruction areas. Used by Innovus for placement and routing. This project uses gsclib090_translated.lef.

Q44: What is a LIB file?

Liberty Timing Library. Contains timing, power, and function information for standard cells: delay tables (NLDM), setup/hold times, leakage power, etc. Used for synthesis and STA. This project uses slow.lib.

Q45: Difference between LEF and LIB?

LEF = PHYSICAL information (cell size, pin positions, metal layers). LIB = TIMING/POWER information (delays, setup/hold, power). Synthesis needs LIB. Physical design needs both LEF and LIB.

Q46: What is useful skew?

Useful skew intentionally introduces clock skew to borrow timing from a path with extra slack and give it to a path with tight slack. CTS tools can exploit useful skew to close otherwise unclosable timing.

Q47: Why is hold independent of clock period?

Hold is checked at the SAME clock edge. The hold equation is: Arrival Time > Hold Time. Clock period does not appear in this equation. Increasing clock period fixes setup but NOT hold.

Q48: What happens if you increase utilization to 80%?

Routing congestion increases dramatically. Placement becomes denser, reducing room for buffers. Timing closure becomes much harder. DRC violations increase. Power density and thermal hotspots worsen.

Q49: What is the difference between placement density and utilization?

Utilization = cell area / core area (global metric). Placement density = local cell density in a region. A design can have 50% global utilization but have local hotspots at 90% density, causing routing congestion.

Q50: How would you optimize power in this design?

Options: (1) Use multi-Vt cells (HVT for non-critical, LVT for critical paths). (2) Clock gating to reduce switching. (3) Power gating for unused blocks. (4) Reduce clock frequency. (5) Lower supply voltage.

Q51: What is OCV?

On-Chip Variation. Manufacturing and environmental variations cause different parts of the same chip to have slightly different delays. OCV analysis adds derating factors to account for this.

Q52: Why CTS after placement, not before?

The clock tree must know WHERE the flip-flops are to build balanced paths. CTS needs accurate wire lengths from placed cells. Running CTS before placement would build a tree based on wrong distances.

Q53: What is the difference between synthesis and STA?

Synthesis CREATES the netlist (transforms RTL into gates, optimizes for timing). STA ANALYZES timing on an already-created netlist without changing it. Synthesis uses internal STA, but standalone STA (Tempus) is more accurate.

Q54: What is clock latency?

Clock latency is the delay from the clock source to a flip-flop clock pin. Source latency = delay from clock generator to chip pin. Network latency = delay from chip pin to flip-flop (through clock tree buffers).

Q55: What is the slow corner library?

The slow corner library models worst-case conditions: high temperature, low voltage, slow process. This gives the longest cell delays, making it hardest to meet setup timing. It is the most conservative library for setup analysis.

Q56: Why was slow.lib used for both early and late analysis?

This is a single-corner analysis (simplified for academic purposes). In industry, early analysis uses the fast corner library and late analysis uses the slow corner library for accurate hold/setup checking.

Q57: What metal layers are used for routing?

In this 90nm library, Metal1 through Metal9 are available. Metal1 is used for standard cell internal routing. Metal2-Metal7 for signal routing. Metal8/Metal9 for power ring. The routing tool decides actual layer assignment.

Q58: What is NanoRoute?

NanoRoute is the routing engine inside Cadence Innovus. It handles both global routing (route planning) and detailed routing (exact track assignment). It supports timing-driven and SI-driven routing modes.

Q59: What is SPEF?

Standard Parasitic Exchange Format. Contains extracted resistance and capacitance of routed wires. Used for accurate post-route STA. Generated by extractRC command after placement.

Q60: What is SDF?

Standard Delay Format. Contains gate delays and interconnect delays. Used for gate-level timing simulation. Generated by write_sdf during synthesis and after routing.

Q61: Why did the cell count DECREASE after DFT (636->623)?

Some cells were merged or optimized during the incremental synthesis step (synthesize -to_mapped -incremental) that runs after DFT scan replacement. The net effect is fewer cells even though scan muxes were added.

Q62: What is a skew group in CTS?

A skew group defines a set of clock sinks that should have balanced (low-skew) clock arrival. In this project, all 36 scan flip-flops are in one skew group (clk/sdc1) because they all share the same clock domain.

Q63: What does set_case_analysis do?

It fixes a port or pin to a constant value for timing analysis. set_case_analysis 0 [get_ports scan_en] tells the STA tool that scan_en=0 (functional mode), so it only analyzes functional paths, not scan shift paths.

Q64: Why is the post-route slack (+59 ps) BETTER than post-CTS repair (+47 ps)?

Routing optimization may reroute critical nets on lower-resistance paths, or the post-route timing engine may use more accurate delay models than the pre-route estimates.

Q65: What is wire load model?

A statistical model of wire parasitic capacitance and resistance based on net fanout. Used during synthesis when real routing is not available. The reports show 'Wireload mode: enclosed' and '<none> (D)' meaning the default library wireload is used.

Q66: Walk through the complete flow from RTL to GDS.

1. Write RTL (ALU.v). 2. Simulate with NCVerilog (tb1/tb2/tb3). 3. Check coverage. 4. Synthesize with Genus (three netlists). 5. Verify with Conformal LEC. 6. Analyze timing with Tempus STA. 7. Insert scan chains (DFT). 8. Re-check timing (post-DFT STA). 9. Floorplan in Innovus. 10. Place cells. 11. Build clock tree (CTS). 12. Optimize CTS. 13. Route signals. 14. Signoff DRC/connectivity. 15. Stream out GDSII.

Q67: What is the most important metric at each stage?

Synthesis: slack and area. LEC: PASS/FAIL. STA: slack (setup and hold). DFT: scan chain length and coverage. Placement: utilization and congestion. CTS: skew and insertion delay. Routing: DRC and timing. Signoff: all clean.

Q68: What would you change for a real tape-out?

Add multi-corner MMMC (fast/slow/typical), add OCV derating, add real input/output delays, increase scan chains for shorter test time, add ESD/IO pad ring, add power grid stripes, run IR drop analysis, run EM analysis, run LVS against schematic.

Q69: What is the difference between congestion and utilization?

Utilization is cell area / core area (global). Congestion is local demand for routing tracks exceeding supply. High utilization causes congestion, but congestion can also happen at low utilization if cells cluster locally.

Q70: How do you choose the clock period?

Based on target application frequency and critical path delay. Start with the longest (most relaxed) period that meets the spec, then tighten if performance allows. The critical path in this design is ~7 ns (divider logic), so 8 ns is achievable and 4 ns requires aggressive optimization.

Q71: What are the 16 operations?

ADD, SUB, MUL, DIV, MOD, AND, NAND, OR, NOR, XOR, XNOR, VEC_REV (bit reverse), PAR (parity), SIGN (MSB check), BIT_EQUAL (equality), AVERAGE.

Q72: Why use parameters for operation encoding?

Parameters make the code readable and self-documenting. ADD = 4'b0000 is clearer than a raw binary value. Parameters are resolved at compile time and have zero hardware cost.

Q73: Why non-blocking assignments (<=)?

Non-blocking assignments model flip-flop behavior in Verilog. They ensure all right-hand sides are evaluated BEFORE any left-hand side is updated, preventing race conditions in sequential logic.

Q74: What is the timescale directive?

``timescale 1ns/1ps` sets the time unit to 1 nanosecond and time precision to 1 picosecond. This means #5 is a 5 ns delay, and timing can be measured to 0.001 ns accuracy.

Q75: Why does the testbench toggle reset multiple times?

The reset sequence (0->1->0->1->0) tests both assertion and de-assertion of reset. It verifies that the ALU correctly clears outputs when reset is high and resumes normal operation when reset goes low.

Q76: What is elaboration?

Elaboration builds the design hierarchy from parsed HDL. It resolves module instantiations, connects ports, propagates parameters, and creates a unified design database. It is the step between parsing and synthesis/simulation.

Q77: What is the SDF file used for?

The SDF (Standard Delay Format) file contains gate and interconnect delays. It is used for gate-level timing simulation (back-annotation). The simulator reads the SDF to annotate delays onto netlist instances for accurate timing behavior.

Q78: Why do write_sdf options include -nonegchecks?

It suppresses negative timing checks from the SDF output. Negative setup/hold times can occur in certain cell corners and confuse some simulators. -nonegchecks prevents this.

Q79: What does -recrem split do?

It splits combined recovery/removal timing checks into separate timing arcs. This makes the SDF compatible with simulators that don't support combined recovery/removal checks.

Q80: What is the Conformal warning about X assignments?

'Converted 16 X assignment(s) as don't care(s)' means 16 scenarios in the RTL produce unknown (X) values (like dividing by zero). Conformal treats these as don't-cares, meaning any revised output is acceptable for those cases.

Q81: What is gsclib090site?

It is the standard cell placement site defined in the LEF library for the 90nm Cadence generic library. The site defines the row height (3.36 um) and minimum width for cell placement.

Q82: What does globalNetConnect do?

It connects all VDD pins of standard cells to the VDD power net, and all VSS pins to the VSS ground net. Without this, cells would not have power connections.

Q83: What does sroute do?

Special routing (sroute) routes power and ground nets. Unlike signal routing, power routing follows special rules (wider wires, follows power grid structure). sroute -nets {VDD VSS} routes both supply rails.

Q84: What does legalizePin do?

It snaps I/O pins to legal positions on the routing grid. Pins must align to metal track positions for the router to connect them without DRC violations.

Q85: Why define scan chains before placement?

The placer uses scan chain information to place scan-connected flip-flops close together, minimizing scan chain wire length. This improves scan shift timing and reduces routing congestion.

Q86: What does extractRC do?

It performs parasitic extraction - computing the resistance (R) and capacitance (C) of routed metal wires. These parasitics are used for accurate post-route timing analysis.

Q87: What is ccopt_pro?

An advanced CTS command that performs DRV (Design Rule Violation) fixing, skew fixing, rebuffering, and placement refinement on the clock tree. It is used after initial CTS to fine-tune the clock network.

Q88: What routing modes does NanoRoute support?

Timing-driven (routes critical paths first), SI-driven (avoids crosstalk), and non-timing-driven (default). This project uses non-timing-driven mode as shown by `routeWithTimingDriven false`.

Q89: What is streamOut?

`streamOut` exports the physical layout in GDSII format, which is the industry-standard file sent to the foundry for mask fabrication. It converts all cells, routing, and vias into geometric shapes on layers.

Q90: What is a DEF file?

Design Exchange Format. Contains the physical layout information: die size, row definitions, placed cell coordinates, routed net geometries, and via information. Used to transfer physical data between tools.

Q91: What is the difference between setup and hold?

Setup: data must be stable BEFORE clock edge. Hold: data must be stable AFTER clock edge. Setup violations are fixed by slowing the data path or increasing the clock period. Hold violations are fixed by adding delay to the data path (buffers).

Q92: Can hold violations be fixed by changing clock period?

NO. Hold is independent of clock period. Hold only depends on the delay between launch and capture edges at the SAME clock cycle. Only adding delay (buffers) on the data path fixes hold.

Q93: What is the role of buffers in the design?

Buffers serve multiple purposes: (1) reduce transition time violations by refreshing signal edges, (2) reduce fanout violations by splitting load, (3) fix hold violations by adding delay, (4) in CTS, balance clock arrival times.

Q94: Why was the balanced netlist chosen for physical design?

The balanced netlist at 125 MHz represents a practical operating frequency with reasonable area and power. The min area netlist cannot meet 8ns timing. The tight timing netlist is unnecessarily large for 125 MHz.

Q95: What is the MMMC file?

Multi-Mode Multi-Corner view definition file. It defines library sets, constraint modes, delay corners, and analysis views. In this project, a single view (`view1`) with `slow.lib` and the balanced DFT constraint is used.

Q96: What is the difference between early and late timing?

Late timing = setup analysis (worst-case slow paths). Early timing = hold analysis (worst-case fast paths). Late uses maximum delays. Early uses minimum delays.

Q97: What would happen if clock uncertainty increased to 1 ns?

The effective timing budget shrinks to $8.0 - 1.0 - 0.108 = 6.892$ ns. The balanced netlist with 6.986 ns data path delay would VIOLATE timing. More aggressive cells or lower uncertainty would be needed.

Q98: What is the relationship between frequency and power?

Dynamic power is proportional to frequency ($P = \alpha * C * V^2 * f$). Doubling frequency doubles dynamic power. The tight timing netlist at 250 MHz shows 2.2x the power of the balanced at 125 MHz.

Q99: How many metal layers are in this technology?

9 metal layers (Metal1 through Metal9). Metal1 is the lowest (for local cell connections), Metal9 is the highest (for power distribution).

Q100: What is the aspect ratio of the die?

Approximately 1:1 (square). Specified by -r 1 in the floorPlan command. A square die provides the most uniform wire lengths and least routing bias.

Q101: What is the difference between core area and die area?

Core area is where standard cells are placed (inside the margins). Die area = core area + margins on all sides. The 4.06 um margins accommodate the power ring.

Q102: What does report_constraints -all_violators show?

It lists all timing constraints that are violated, including setup, hold, max capacitance, max transition, and max fanout violations with their slack values.

Q103: Why use rising edge for all scan flip-flops?

Using a single clock edge simplifies CTS because only one tree is needed. Mixing edges (set dft_mix_clock_edges_in_scan_chains true) is allowed for flexibility but this design only has rising-edge flip-flops.

Q104: What is the purpose of the VCD file?

Value Change Dump. It records every signal transition during simulation. Used by waveform viewers (SimVision) for debugging and by coverage tools for toggle coverage analysis.

Q105: What does \$dumpvars(0, tb1) do?

It dumps all signals in the tb1 hierarchy (0 = dump all levels of hierarchy) to the VCD file. This enables complete waveform viewing of all internal ALU signals.

Q106: What is the operating condition in the reports?

'slow (balanced_tree)' indicates the slow process corner with a balanced wire load tree topology. This is the worst-case corner for setup timing analysis.

Q107: Why is leakage power percentage higher in min area?

Min area has 7.86% leakage vs 2.87% in tight timing. Although tight timing has more leakage in absolute terms (32.73 vs 19.25 uW), its much higher dynamic power (1109 vs 225 uW) reduces leakage's relative share.

Q108: What is the fan-out in the design?

The QoR report shows: Max Fanout = 36 (the clk signal, which drives all 36 FFs), Min Fanout = 1, Average Fanout = 2.5.

Q109: What is a via stack?

A via stack connects different metal layers vertically. The power ring uses a via stack from Metal1 to Metal9 (-stacked_via_bottom_layer Metal1 -stacked_via_top_layer Metal9) to connect the ring to cell power pins.

Q110: What is the difference between addRing and addStripe?

addRing creates a continuous ring of power/ground around the die perimeter. addStripe creates internal parallel power/ground lines across the core area. This project only uses addRing (no internal stripes).

Q111: Is this design manufacturable?

Nearly. It has 0 connectivity violations and timing is clean. The 12 DRC violations are power ring edge effects that would need minor cleanup. In a production flow, these would be fixed before tape-out.

Q112: What is the maximum operating frequency?

Based on the post-route critical path delay of 7.255 ns and required timing margin (setup + uncertainty = 0.687 ns), the maximum frequency is approximately $1/(7.255+0.687) = 126$ MHz.

Q113: What library format is used?

Liberty (.lib) for timing, LEF for physical layout, Verilog (.v) for cell functional models (used by LEC).

Q114: Why is there no default case in the RTL case statement?

All 16 combinations of sel_ff (4'b0000 to 4'b1111) are explicitly covered. A default case is unnecessary. The synthesis tool can infer that no other value is possible, avoiding latches.

Q115: What is the total number of key interview points?

Major: (1) One-cycle pipeline latency, (2) Critical path through divider, (3) Area-timing-power trade-off, (4) LEC exhaustive vs simulation finite, (5) Hold independent of period, (6) CTS fixes post-placement violations, (7) Scan mux adds delay, (8) 50% utilization rationale, (9) Power ring on top metals, (10) Buffer insertion for timing fix.

Q116: What would you do differently in a real project?

Multi-corner MMMC analysis, OCV derating, IO pad ring, ESD protection, power grid IR drop analysis, electromigration analysis, DFM (Design for Manufacturing) rules, formal verification of DFT, ATPG pattern generation and fault simulation.

Q117: What is the technology node?

90nm Cadence generic standard cell library (gsclib090). This is an academic/educational library, not a production foundry PDK.

Q118: Why is the worst hold path after routing through scan_en?

The scan_en port connects directly to scan flip-flop SE pins with minimal routing delay. Since no input delay is specified, the data arrives instantly (0 ps) while the hold requirement is ~6 ps, creating a very tight margin that CTS optimization fixes to +10 ps.

Q119: What Genus version was used?

Genus(TM) Synthesis Solution 19.13-s073_1, as reported in all synthesis reports.

Q120: What Innovus version was used?

Cadence Innovus 20.10-p004_1, as reported in all physical design reports.

18. Appendix

18.1 Glossary

Term	Definition
ASIC	Application-Specific Integrated Circuit
RTL	Register Transfer Level - behavioral hardware description
GDS/GDSII	Graphic Data System - layout file format for fabrication
SDC	Synopsys Design Constraints - timing constraint format
STA	Static Timing Analysis
LEC	Logic Equivalence Checking
DFT	Design for Testability
CTS	Clock Tree Synthesis
DRC	Design Rule Check
LVS	Layout Versus Schematic
LEF	Library Exchange Format - physical cell information
LIB	Liberty format - timing/power cell information
SDF	Standard Delay Format - delay annotation file
SPEF	Standard Parasitic Exchange Format
DEF	Design Exchange Format - physical layout data
MMMC	Multi-Mode Multi-Corner analysis
GBA	Graph-Based Analysis (conservative STA)
PBA	Path-Based Analysis (accurate STA)
OCV	On-Chip Variation
ATPG	Automatic Test Pattern Generation
TNS	Total Negative Slack
WNS	Worst Negative Slack
QoR	Quality of Results
VCD	Value Change Dump (waveform format)
HVT	High Voltage Threshold (slow, low-leakage cell)
LVT	Low Voltage Threshold (fast, high-leakage cell)
SVT	Standard Voltage Threshold
IR Drop	Voltage drop across power grid resistance
EM	Electromigration - wire reliability concern
Fanout	Number of inputs driven by a single output
Slack	Timing margin (positive=met, negative=violated)
Skew	Difference in clock arrival time at two flip-flops

18.2 Important Formulas

Formula	Equation
Setup Slack	Clock Period - Setup Time - Uncertainty - Data Path Delay
Hold Slack	Data Path Delay - Hold Time
Required Time (Setup)	Clock Period - Setup Time - Uncertainty
Dynamic Power	$\alpha \times C_{load} \times VDD^2 \times \text{frequency}$
Leakage Power	$I_{off} \times VDD$
Utilization	Cell Area / Core Area
Aspect Ratio	Height / Width
Max Frequency	$1 / (\text{Data Arrival} + \text{Setup} + \text{Uncertainty})$

18.3 File Structure

Directory	Contents
rtl/	ALU.v (RTL source)
testbench/	tb1.v, tb2.v, tb3.v (testbenches)
synthesis_constraints/	design_min_area.sdc, design_balanced.sdc, design_tight_timing.sdc
synthesis_scripts/	synth_min_area.tcl, synth_balanced.tcl, synth_tight_timing.tcl
synthesis_outputs/	Netlists, SDF, SDC, reports for all three cases
lec_scripts/	LEC .do scripts for all netlists + bad cases
lec_bad_netlists/	Intentionally corrupted netlists + documentation
lec_outputs/	LEC logs for all cases
sta_scripts/	STA TCL scripts for all three netlists + experiments
sta_reports/	STA timing reports for all cases
dft_scripts/	DFT scan insertion script
dft_reports/	DFT netlists, scan chain reports, area/timing/power
post_dft_sta_scripts/	Post-DFT STA scripts
post_dft_sta_reports/	Post-DFT timing reports
pd_scripts/	Innovus TCL scripts for all PD stages
pd_reports/	All PD reports (placement, CTS, routing, signoff)
pd_snapshots/	Innovus database snapshots at each stage

18.4 Key Cadence Commands Reference

Tool	Command	Purpose
Genus	read_hdl	Read Verilog RTL or netlist
Genus	elaborate	Build design hierarchy
Genus	read_sdc	Read timing constraints
Genus	syn_generic	Technology-independent optimization
Genus	syn_map	Technology mapping to standard cells
Genus	write_hdl	Export gate-level netlist
Genus	report_timing	Show worst timing path
Tempus	read_lib	Read Liberty timing library
Tempus	read_verilog	Read gate-level netlist
Tempus	set_top_module	Set top module for analysis
Tempus	report_timing	Generate timing report
Tempus	report_timing -early	Generate hold timing report
Conformal	read_library	Load standard cell models
Conformal	read design -golden	Read reference design
Conformal	read design -revised	Read implementation
Conformal	set system mode lec	Enter equivalence checking mode
Conformal	compare	Run formal comparison
Innovus	init_design	Initialize design database
Innovus	floorPlan	Create floorplan
Innovus	placeDesign	Place standard cells
Innovus	cconst_design	Build clock tree
Innovus	optDesign	Optimize timing
Innovus	routeDesign	Route signal wires
Innovus	verify_drc	Check design rules
Innovus	streamOut	Export GDSII layout

19. Detailed Script Analysis

This chapter provides a line-by-line analysis of every TCL script used in the project. Understanding these scripts is critical for interviews because interviewers expect you to explain not just WHAT was done, but HOW.

19.1 Synthesis Script Deep Dive (synth_balanced.tcl)

The synthesis script follows a standard Genus flow. Here is every command explained in detail:

19.1.1 Directory Creation

```
if {[file exists synthesis_outputs]} {
    file mkdir synthesis_outputs
}
```

This TCL conditional checks if the output directory exists. If not, it creates it. This prevents Genus from crashing when trying to write output files to a non-existent directory. This is a defensive programming practice used in all production scripts.

19.1.2 Search Path Configuration

```
set_db lib_search_path ./lib
set_db hdl_search_path ./rtl
```

set_db is the Genus command for setting database attributes. lib_search_path tells Genus WHERE to find technology library files (.lib). hdl_search_path tells Genus WHERE to find HDL source files (.v). These paths are relative to the directory where Genus is invoked.

NOTE: In Synopsys Design Compiler, the equivalent commands would be: set search_path [list ./lib] and set_app_var search_path ./rtl. Be prepared to explain both syntaxes in interviews.

19.1.3 Library Loading

```
set_db library slow.lib
```

This loads the Liberty timing library. The file slow.lib contains timing models (delay tables), power models (leakage and dynamic), function definitions (Boolean equations), and physical constraints (area, pin capacitance) for every standard cell in the 90nm library. The 'slow' indicates this is the slow process corner (worst-case for setup timing).

A Liberty library entry for a cell like NAND2XL would include: cell delay as a function of input transition and output load (NLDM table), setup and hold times for sequential cells, cell area in library units, pin capacitance for each input, and the Boolean function the cell implements.

19.1.4 Write SDF Options

```
write_sdf \
-timescale ns \
-nonegchecks \
-recrem split \
-edges check_edge \
> synthesis_outputs/balanced_case/delays_balanced.sdf
```

Each option explained:

Option	Purpose
-timescale ns	Sets the time unit in the SDF file to nanoseconds
-nonegchecks	Removes negative timing check values that can confuse simulators
-recrem split	Splits combined recovery/removal checks into separate arcs for simulator compatibility

Option	Purpose
-edges check_edge	Includes edge information (rise/fall) in timing checks for accuracy
> file.sdf	Redirects output to the specified file path

19.2 LEC Script Deep Dive (lec_balanced.do)

The LEC script uses Conformal's command language (not standard TCL). Here is every command:

19.2.1 Library and Design Loading

```
read library lib/slow.v -verilog -both
```

This reads the Verilog functional models of standard cells. The -both flag loads the library into BOTH the golden and revised design containers. This is necessary because the revised netlist uses standard cell instances that need to be resolved.

NOTE: LEC uses slow.v (Verilog cell models) NOT slow.lib (Liberty timing). LEC does not need timing information - it only needs the Boolean functions of each cell to verify logical equivalence.

```
read design rtl/ALU.v -verilog -golden
```

```
read design synthesis_outputs/balanced_case/synthesized_netlist_balanced.v -verilog -revised
```

The -golden flag loads the design as the reference. The -revised flag loads it as the implementation. LEC will prove that revised implements the same Boolean function as golden at every compare point.

19.2.2 Comparison Execution

```
set system mode lec
add compared point -all
compare
```

set system mode lec transitions from setup mode to LEC mode. This triggers key point mapping. add compared point -all adds all mappable POs and DFFs to the comparison list. compare invokes the formal verification engines (BDD, SAT solvers) to exhaustively prove equivalence.

19.3 STA Script Deep Dive (sta_min_area.tcl)

The STA script for the min area netlist is the most documented. Key insights:

19.3.1 Why All Three Netlists Use the 8 ns Constraint

All three STA scripts read design_balanced.sdc (8 ns period), not the original synthesis SDC. This is intentional. By using the SAME constraint, the STA reveals how each netlist performs against the SAME target, making the comparison fair. The min area netlist fails because it was built for 15 ns but tested at 8 ns.

19.3.2 PBA vs GBA Explanation

Each STA script generates both GBA and PBA reports. Here is why both matter:

GBA (Graph-Based Analysis) computes the worst-case slew at each node by merging all possible arriving signals. This is conservative (pessimistic) because a node might see a slow slew from path A and a fast slew from path B, but GBA assumes the worst case for both.

PBA (Path-Based Analysis) traces a specific timing path and propagates the actual slew along that path. This gives a more accurate (less pessimistic) slack value. The PBA options used are:

Option	Meaning
-retime path_slew_propagation	Enable path-based slew propagation instead of graph-based
-max_path 50	Report up to 50 worst timing paths

Option	Meaning
-nworst 50	Consider 50 worst paths per endpoint
-path_type full_clock	Show complete path including launch and capture clock segments

19.3.3 check_timing Report Analysis

The check_timing report for the balanced netlist shows these warnings:

Warning	Count	Meaning	Impact
ideal_clock_waveform	1	Clock is ideal (not propagated through buffers)	Expected pre-CTS
no_drive	22	22 input ports have no driving cell specified	Input transition used instead
no_input_delay	21	21 inputs have no input delay constraint	Inputs assumed synchronous
uncons_endpoint	16	16 output endpoints are unconstrained	No output delay specified

These are all expected for this project setup. In industry, all of these would be constrained. The no_input_delay warning explains why hold violations appear on input paths.

19.4 DFT Script Deep Dive (dft_balanced.tcl)

The DFT script uses Genus legacy UI syntax (forced by set_db common_ui false). Key commands:

19.4.1 DFT Configuration

```
set_attribute dft_scan_style muxed_scan /
define_dft shift_enable -active high -create_port scan_en
define_dft test_clock clk
```

set_attribute dft_scan_style muxed_scan selects mux-based scan (vs. level-sensitive scan latch, or clocked scan). Mux scan is the most common style in industry. define_dft shift_enable creates the scan_en port and defines it as active-high. define_dft test_clock specifies that clk is used for shifting patterns during test.

19.4.2 DFT Rule Checking and Violation Fixing

```
check_dft_rules > $report_dir/dft_rules_balanced.rep
fix_dft_violations -test_control scan_en -async_set -async_reset -clock
```

check_dft_rules identifies any flip-flops that cannot be made scannable (e.g., flip-flops with asynchronous set/reset that conflict with scan operation). fix_dft_violations fixes these by adding test-mode control logic: the -async_set, -async_reset, and -clock options tell the tool to add scan_en-controlled logic to prevent asynchronous signals from interfering during scan shift.

19.4.3 Scan Chain Connection

```
set_attribute dft_min_number_of_scan_chains 1 [find / -design *]
set_attribute dft_mix_clock_edges_in_scan_chains true [find / -design *]
connect_scan_chains -auto_create_chains -preview [find / -design ALU]
connect_scan_chains -auto_create_chains [find / -design ALU]
```

dft_min_number_of_scan_chains 1 tells the tool to create at minimum 1 scan chain. With 36 FFs, one chain of length 36 is created. dft_mix_clock_edges_in_scan_chains true allows the chain to mix rising and falling edge FFs (though all FFs here are rising edge). The -preview flag first shows what the tool would do without actually modifying the netlist. The second call (without -preview) performs the actual scan stitching.

INTERVIEW TIP: If asked 'How many scan chains should you have?', answer: More chains = shorter shift time but more pins (each chain needs SI and SO). Industry rule of thumb: chain length of 200-500 FFs is typical. Our chain of 36 is very short, appropriate for this small design.

19.5 Innovus Floorplan Script Deep Dive (floorplan_05.tcl)

19.5.1 Design Initialization Variables

```
set init_top_cell      "ALU"
set init_verilog       "dft_reports/balanced/dft_netlist_balanced.v"
set init_lef_file      [list "lef/gsclib090_translated.lef"]
set init_gnd_net       "VSS"
set init_pwr_net       "VDD"
set init_mmmc_file     "pd_scripts/alu_mmmc_views.view"
set init_io_file       "pd_scripts/pin_location.io"
```

Each variable configures Innovus before design initialization:

Variable	Value	Purpose
init_top_cell	ALU	Names the top module for PD
init_verilog	dft_netlist_balanced.v	Post-DFT scan-inserted netlist (NOT original RTL)
init_lef_file	gsclib090_translated.lef	Physical cell library (dimensions, pins, layers)
init_gnd_net	VSS	Ground power net name
init_pwr_net	VDD	Supply power net name
init_mmmc_file	alu_mmmc_views.view	Multi-corner multi-mode timing configuration
init_io_file	pin_location.io	I/O pin placement assignments

19.5.2 Power Ring Parameters Explained

The addRing command parameters:

Parameter	Value	Meaning
-center 1	1 (true)	Center the ring between core edge and die edge
-stacked_via_top_layer Metal9	Metal9	Top layer for via connections between ring segments
-stacked_via_bottom_layer Metal1	Metal1	Bottom layer for via connections to cell power pins
-type core_rings	core_rings	Ring type: surrounds the core area
-jog_distance 0.435	0.435 um	Maximum jog allowed to avoid obstructions
-threshold 0.435	0.435 um	Minimum distance to avoid ring breaks
-nets {VSS VDD}	VSS, VDD	Create rings for both ground and power
-follow core	core	Ring follows the core boundary shape
-layer {bottom Metal8 top Metal8 right Metal9 left Metal9}	Metal8/Metal9	Horizontal segments on Metal8, vertical on Metal9
-width 1.25	1.25 um	Width of each ring wire
-spacing 0.4	0.4 um	Spacing between VDD and VSS rings

19.6 Innovus Placement Script Deep Dive (placement.tcl)

Key placement commands and their effects:

19.6.1 Scan Chain Definition Before Placement

```
specifyScanChain scan1 -start DFT_sdi_1 -stop DFT_sdo_1
specifyScanChain scan2 -start DFT_sdi_2 -stop DFT_sdo_2
```

These commands inform the placer about scan chain topology. The placer will try to place consecutive scan flip-flops close together to minimize scan chain wire length. Note: scan2 is specified even though only one scan chain exists - this may be a template from a multi-chain flow.

19.6.2 Post-Placement Analysis Commands

```
extractRC
rcOut -spef pd_reports/placement_reports/alu_placement.spef
write_sdf -ideal_clock_network pd_reports/placement_reports/physical_design_alu_placed.sdf
```

extractRC performs parasitic extraction: it computes the resistance and capacitance of every net based on the placed cell locations and estimated routing paths (trial routing). rcOut writes these parasitics to a SPEF file. write_sdf -ideal_clock_network exports an SDF file with ideal clock (zero clock delay) for timing verification.

19.7 Innovus CTS Script Deep Dive (cts.tcl)

The CTS flow in this project uses Innovus's modern CCOpt (Clock Concurrent Optimization) engine:

19.7.1 Buffer Cell Selection

```
set_ccopt_property buffer_cells {CLKBUF3 CLKBUF2 CLKBUF8 CLKBUF12 CLKBUF16 CLKBUF20
CLKINV3 CLKINV2 CLKINV8 CLKINV12 CLKINV16}
```

This specifies which cells CCOpt is allowed to use for clock tree construction. The list includes multiple drive strengths (X2 through X20) of both clock buffers (CLKBUF*) and clock inverters (CLKINV*). CCOpt selects the optimal cell at each tree level based on the load and transition requirements. Larger buffers are used near the root (high fanout), smaller ones near leaves.

19.7.2 Clock Tree Specification

```
create_ccopt_clock_tree_spec -file pd_scripts/ccopt_alu.spec -keep_all_sdc_clocks -views
{view1}
source pd_scripts/ccopt_alu.spec
ccopt_design -check_prerequisites
ccopt_design
```

create_ccopt_clock_tree_spec generates a clock tree specification file from the SDC clock definitions. -keep_all_sdc_clocks ensures no clocks are dropped. source loads the generated spec. ccopt_design -check_prerequisites verifies the design is ready for CTS. ccopt_design builds the actual clock tree.

The generated ccopt_alu.spec file reveals: clock source = port clk, clock period = 8 ns, target max transition = 0.4 ns (from SDC), and one skew group (clk/sdc1) containing all 36 scan flip-flops.

19.8 Innovus Routing Script Deep Dive (routing.tcl)

19.8.1 NanoRoute Configuration

```
setNanoRouteMode -quiet -timingEngine {}
setNanoRouteMode -quiet -routeWithSiPostRouteFix 0
setNanoRouteMode -quiet -routeWithTimingDriven false
setNanoRouteMode -quiet -routeWithSiDriven false
```

These settings configure the NanoRoute routing engine:

Setting	Value	Meaning
-timingEngine {}	empty	Use default timing engine
-routeWithSiPostRouteFix 0	disabled	No post-route signal integrity (crosstalk) fixing

Setting	Value	Meaning
-routeWithTimingDriven false	disabled	Non-timing-driven routing (routes all nets equally)
-routeWithSiDriven false	disabled	No signal integrity-driven routing

NOTE: In a production flow, timing-driven routing (routeWithTimingDriven true) is almost always used. It prioritizes critical path nets, giving them shorter, more direct routes. Non-timing-driven routing is faster but may require more post-route optimization.

20. Detailed Report Analysis

This chapter extracts and explains the numerical data from every key report generated during the flow.

20.1 Synthesis Area Report Breakdown

The area reports provide a breakdown by cell type category:

Category	Min Area (um2)	Min Area %	Balanced (um2)	Balanced %	Tight (um2)	Tight %
Sequential	718.298	19.2%	736.464	17.5%	744.790	11.8%
Inverter	52.226	1.4%	161.220	3.8%	423.107	6.7%
Buffer	0.000	0.0%	4.541	0.1%	210.418	3.3%
Logic (comb)	2975.374	79.4%	3296.299	78.5%	4946.341	78.2%
Physical	0.000	0.0%	0.000	0.0%	0.000	0.0%
TOTAL	3745.898	100%	4198.524	100%	6324.656	100%

Key observations: (1) Inverter area grows 8x from min area to tight timing because the tool inserts inverters for signal polarity management and fanout splitting. (2) Buffer area grows from 0 to 210 um2 as the tool adds buffers for timing. (3) Logic area grows 66% as cells are upsized to higher drive strengths. (4) Sequential area barely changes because the flip-flop count is fixed at 36.

20.2 Cell Count Analysis

Top 10 most-used cell types in each netlist:

Cell Type	Min Area	Balanced	Tight	Function
NAND2XL	61	87	142	2-input NAND, minimum size
NOR2XL	44	65	127	2-input NOR, minimum size
AND2X1	48	54	58	2-input AND, X1 drive
ADDFXL	36	36	24	Full adder, minimum size
CLKINVX1	16	35	74	Clock inverter, X1 drive
ADDFX1	39	19	20	Full adder, X1 drive
MXI2XL	15	10	21	2:1 MUX inverting, minimum
NAND2BX1	4	15	21	2-input NAND with one inverted
OAI21X1	6	33	72	OR-AND-INVERT 2-1
AOI21X1	15	40	65	AND-OR-INVERT 2-1

The tight timing netlist uses many more OAI21X1 and AOI21X1 cells because these compound gates (OR-AND-INVERT, AND-OR-INVERT) implement two logic levels in a single cell, reducing the critical path depth.

20.3 Power Report Analysis

Power breakdown across all three netlists (all values in microwatts):

Power Component	Min Area	Balanced	Tight Timing	Explanation
Leakage	19.25 (7.86%)	20.56 (4.01%)	32.73 (2.87%)	Static power from transistor off-state current. Proportional

Power Component	Min Area	Balanced	Tight Timing	Explanation
				to total transistor width.
Internal	100.07 (40.88%)	214.29 (41.75%)	560.48 (49.06%)	Power consumed inside cells during switching. Proportional to internal node capacitance and frequency.
Switching	125.45 (51.25%)	278.39 (54.24%)	549.15 (48.07%)	Power for charging/discharging output load capacitances. Proportional to load, V_{DD}^2 , and frequency.
Clock	3.29 (1.35%)	6.22 (1.21%)	12.49 (1.09%)	Power consumed by clock network distribution.
TOTAL	244.78	513.24	1142.36	Total power consumption

The VDD voltage is 0.9V (from Innovus power reports). Using the formula $P_{dynamic} = \alpha * C * V^2 * f$, doubling frequency from 66.67 MHz (min area) to 125 MHz (balanced) approximately doubles switching power (125.45 -> 278.39 uW), confirming the linear frequency dependence.

20.4 Timing Report Reading Guide

Understanding how to read a Genus/Tempus timing report is essential for interviews. Here is a column-by-column explanation using the balanced netlist timing report:

Column	Meaning	Example Value
Timing Point	The cell instance pin on the path	Y_reg[5]/CK
Flags	<<< marks the endpoint	<<<
Arc	The timing arc through the cell (input -> output)	CK->Q
Edge	Signal direction: R (rise) or F (fall)	F
Cell	Standard cell type from the library	DFFQX1
Fanout	Number of downstream loads driven by this pin	22
Load (fF)	Total capacitive load on the output in femtofarads	41.8
Trans (ps)	Output transition time (slew) in picoseconds	300
Delay (ps)	Cell delay for this specific arc	581
Arrival (ps)	Cumulative arrival time at this pin	581

The critical path of the balanced netlist starts at Y_reg[5]/CK (launch flip-flop) and ends at out1_reg[2]/D (capture flip-flop). The data path delay is 6943 ps. The required time is 7492 ps (8000 - 108 setup - 400 uncertainty). Slack = 7492 - 6943 = 549 ps (MET).

20.5 Physical Design Area Progression

Stage	Cell Count	Total Area (um2)	Buffer Area (um2)	Sequential Area (um2)
Synthesis (balanced)	636	4198.524	4.541	736.464
Post-DFT	623	4331.739	0.000	905.252
Post-Placement	623	4331.739	0.000	905.252
Post-CTS	658	4554.267	313.357	762.955
Post-CTS Repair	661	4567.891	326.981	762.955
Post-Route (Final)	661	4567.891	326.981	762.955

Key transitions: (1) DFT adds scan muxes, increasing sequential area by 23% but reducing cell count due to optimizations. (2) CTS adds 35 buffer cells (313 um2) for clock tree distribution and timing optimization. (3) CTS repair adds 3 more buffers. (4) No additional changes during routing.

20.6 Physical Design Timing Progression (Hold)

Stage	Worst Hold Slack	Status	Endpoint
Post-Placement	-228 ps	VIOLATED	out2_reg[7]/SI (scan path)
Post-CTS	Data in reports	Being fixed	Various
Post-CTS Optimization	Fixed by hold buffers	MET	Various
Post-Route (Final)	+10 ps	MET	out1_reg[0]/SE

Hold violations are systematically fixed during CTS optimization. The optDesign -postCTS -hold command inserts delay buffers on short paths to ensure minimum data arrival time exceeds the hold time requirement.

21. Physical Design Detailed Walkthrough

21.1 Metal Layer Usage

The 90nm library provides 9 metal layers. Here is how they are used in this design:

Layer	Primary Use	Direction	Characteristics
Metal1	Standard cell internal routing, local connections	Horizontal	Thinnest, highest resistance, highest density
Metal2	Signal routing (first global layer)	Vertical	Standard width
Metal3	Signal routing	Horizontal	Standard width
Metal4	Signal routing	Vertical	Standard width
Metal5	Signal routing	Horizontal	Standard width
Metal6	Signal routing	Vertical	Standard width
Metal7	Signal routing	Horizontal	Standard width
Metal8	Power ring (horizontal segments)	Horizontal	Wide, low resistance
Metal9	Power ring (vertical segments)	Vertical	Widest, lowest resistance

Metal layers alternate between horizontal and vertical preferred routing directions. This alternation ensures efficient routing in both dimensions. Higher layers are thicker and wider, making them suitable for power distribution due to lower resistance.

21.2 Design Rule Violations Analysis

DRC violations tracked through the physical design flow:

Stage	Total Violations	Metal1	Metal2	Via2	Types
Post-Placement	995	45	938	12	Margin: 884, outOfDie: 99, AdjCut: 12
Post-Route Signoff	12	12	0	0	outOfDie: 8, Short: 4

The 995 post-placement violations are expected because no signal routing exists yet. Metal2 margin violations (884) occur from trial routing estimates. Via2 adjacent cut violations (12) are from power via arrays. After proper routing, violations drop dramatically to 12. The remaining 12 are power ring edge effects at die boundary.

The 4 Metal1 short violations are at VDD/VSS power rail junctions where the sroute command creates connections between the power ring and standard cell power rails. These could be fixed with minor manual routing adjustment.

21.3 Connectivity Verification

Both post-placement and post-route connectivity checks pass with 0 violations and 0 warnings. The one info message 'Net in1[7]: Found a geometry with bounding box outside the design boundary' is an advisory about a pin placement near the die edge, not a functional problem.

21.4 Gate Count Summary

The post-placement gate count report shows: 1907 equivalent gates in 623 cell instances. The 'equivalent gate' count normalizes all cells to a reference gate area (2.2707 μm^2 , which is the area of a 2-input NAND gate). This means the design complexity is equivalent to approximately 1907 NAND2 gates.

21.5 Supply Voltage

The Innovus power reports confirm the supply voltage: $VDD = 0.9V$. This is the nominal voltage for the 90nm generic library slow corner. The slow corner typically models worst-case conditions: 0.9V supply (reduced from nominal), high temperature (125C or similar), and slow process.

22. STA Experiments Deep Dive

The parametric STA experiments provide critical insight into how constraint changes affect timing. This section analyzes each experiment.

22.1 Clock Slew Sweep (Min Area Netlist at 8 ns)

Clock Slew	Setup Slack	Data Arrival	Setup Time	Required Time
0.0 ns (ideal edge)	-5.215 ns	12.738 ns	0.077 ns	7.523 ns
0.4 ns (baseline)	-5.220 ns	12.809 ns	0.012 ns	7.588 ns
0.8 ns (slow edge)	-5.218 ns	12.847 ns	-0.030 ns	7.630 ns

Analysis: The setup slack barely changes across slew sweeps. This is because:

- The critical path is register-to-register (Y_reg -> divider logic -> out1_reg), so clock slew affects both launch and capture equally.
- A faster clock edge (0 ns) decreases arrival time (12.738 vs 12.809) but also increases setup time requirement (0.077 vs 0.012), partially canceling out.
- A slower clock edge (0.8 ns) increases arrival time (12.847) but actually reduces setup time (to -0.030 ns, meaning the flip-flop is more tolerant), again partially canceling.

INTERVIEW TIP: If asked 'Does clock slew affect setup slack?', the answer is nuanced: for register-to-register paths, clock slew affects both sides equally and largely cancels out. For input-to-register or register-to-output paths, clock slew has more impact.

22.2 Output Load Sweep (Min Area Netlist at 8 ns)

Output Load	Setup Slack	Data Arrival	Note
0.0 pF	-5.220 ns	12.809 ns	Same as baseline
1.0 pF (baseline)	-5.220 ns	12.809 ns	Same as baseline
2.0 pF	-5.220 ns	12.809 ns	Same as baseline

The output load has ZERO effect on setup slack. This is because the critical path endpoint is out1_reg[2]/D, which is a register D-pin, not a primary output. The output load only affects the delay from the register Q-output to the primary output port, which is a separate timing path. The critical path is entirely internal.

This is an important insight: output load affects output-constrained paths, but if the critical path is internal (register-to-register), output load has no impact on the worst slack.

22.3 Post-DFT Slew and Load Matrix

The post-DFT experiments run a 3x3 matrix of slew (0.0/0.4/0.8 ns) and load (0.0/1.0/2.0 pF) combinations. These provide similar insights to the pre-DFT experiments but with the DFT netlist.

23. Industry Best Practices vs. This Project

This chapter compares the academic project methodology with industry best practices, helping interview candidates understand what would change in a real tape-out.

Aspect	This Project	Industry Practice
Corner Analysis	Single corner (slow)	Multi-corner: slow, fast, typical (at least 3)
OCV Analysis	Not used	Always applied with derating factors
Input/Output Delays	Not specified	Always specified from system-level timing budgets
Driving Cell	Not specified (input transition used)	Always specified with <code>set_driving_cell</code>
Clock Gating	Not used	Standard for power reduction (typically 30-50% power savings)
Multi-Vt	Single Vt	Multi-Vt: HVT on non-critical paths, LVT/SVT on critical
IO Pad Ring	No pads	Complete IO ring with ESD protection
IR Drop	Not analyzed	Mandatory for signoff
Electromigration	Not analyzed	Mandatory for signoff
DFM	Not applied	Metal fill, via doubling, OPC
Formal DFT	Not verified	ATPG fault simulation with target >95% fault coverage
Scan Chains	1 chain of 36	Multiple shorter chains (200-500 FFs each)
LVS	Not run	Mandatory layout vs. schematic check
Timing-Driven Routing	Disabled	Always enabled for critical designs
Power Stripes	Ring only	Ring + internal horizontal/vertical stripes
Signoff STA	Innovus internal	Standalone Tempus/PrimeTime signoff

INTERVIEW TIP: In interviews, knowing the gaps between academic and industry flows shows maturity. If asked 'What would you do differently?', recite this table. The top 5 items to mention: multi-corner, OCV, IO constraints, LVS, and IR drop analysis.

24. Common Interview Mistakes to Avoid

Based on common interviewer expectations, here are mistakes candidates make when discussing this type of project:

24.1 Technical Mistakes

- Saying 'hold violation is fixed by increasing clock period.' WRONG. Hold is independent of period. Hold is fixed by adding delay buffers.
- Confusing LEF and LIB. LEF = physical (cell dimensions, pins). LIB = timing and power (delay tables, setup/hold, leakage).
- Saying 'LEC uses simulation.' WRONG. LEC uses formal methods (BDD/SAT). It is exhaustive, not simulation-based.
- Confusing compile_ultra (Synopsys DC) with syn_generic/syn_map (Genus). These are different tools with different command sets.
- Saying 'CTS is done before placement.' WRONG. Placement must come first so CTS knows where the flip-flops are.
- Forgetting that scan insertion adds area AND delay. The scan mux is not free.
- Saying 'higher utilization is better.' WRONG. Higher utilization causes routing congestion, timing closure problems, and thermal issues.

24.2 Presentation Mistakes

- Not knowing actual numbers. Always know: clock period (8 ns), slack values (+59 ps final), area (4567.891 um²), cell count (661), FF count (36).
- Not understanding the critical path. Know that it goes through the divider logic, and explain WHY (division requires iterative subtraction).
- Not being able to walk through the complete flow from memory. Practice: RTL -> Sim -> Coverage -> Synth -> LEC -> STA -> DFT -> Post-DFT STA -> Floorplan -> Place -> CTS -> Route -> Signoff -> GDS.
- Not understanding the one-cycle latency. This is the first question many interviewers ask about the RTL.

24.3 Conceptual Mistakes

- Not understanding the area-timing-power trade-off. Faster = bigger = more power. There is no free lunch.
- Not understanding why timing DEGRADES after placement (real parasitics) and RECOVERS after CTS (balanced clock tree).
- Not understanding the difference between ideal clock (pre-CTS) and propagated clock (post-CTS).
- Not knowing what clock uncertainty models (jitter, OCV, skew).

25. Quick Reference Cards

Tear-out reference cards for interview preparation.

25.1 Numbers You Must Know

What	Value
ALU data width	8-bit inputs, 8-bit dual outputs
Number of operations	16 (sel = 4-bit)
Pipeline latency	1 clock cycle
Flip-flop count	36 (8+8+4+8+8)
Target frequency (balanced)	125 MHz (8 ns)
Technology node	90 nm
Supply voltage	0.9 V
Balanced area (synthesis)	4198.524 um ²
Final area (post-route)	4567.891 um ²
Final cell count	661
Final setup slack	+59 ps
Final hold slack	+10 ps
Die size	~103 x 99 um
Utilization	50%
Scan chains	1, length 36
DRC violations (signoff)	12
Connectivity violations	0
LEC result	PASS (all 3 netlists)

25.2 Tool Command Quick Reference

What You Want	Genus Command	Innovus Command
Read library	set_db library slow.lib	(via MMMC view file)
Read design	read_hdl ALU.v	init_design
Set top module	elaborate ALU	set_top_module ALU
Read constraints	read_sdc design.sdc	(via MMMC view file)
Synthesize	syn_generic; syn_map	N/A
Place cells	N/A	placeDesign
Build clock tree	N/A	cchopt_design
Optimize timing	N/A	optDesign -postCTS
Fix hold	N/A	optDesign -postCTS -hold
Route wires	N/A	routeDesign -globalDetail
Check DRC	N/A	verify_drc
Check connectivity	N/A	verifyConnectivity
Export GDS	N/A	streamOut file.gds
Export netlist	write_hdl > net.v	saveNetlist net.v
Report timing	report timing	report_timing
Report area	report area	report_area -detail
Report power	report power	report_power

25.3 Flow Dependency Chain

Each stage depends on outputs from the previous stage:

RTL (ALU.v) -> [Genus] -> Netlist (.v) + SDC + SDF

Netlist -> [Conformal] -> PASS/FAIL

Netlist + SDC -> [Tempus] -> Timing Reports

Netlist -> [Genus DFT] -> DFT Netlist + SCANDEF + ATPG

DFT Netlist + SDC -> [Tempus] -> Post-DFT Timing

DFT Netlist + LEF + LIB + SDC + IO -> [Innovus] -> GDSII

25.4 Critical Path Summary

The critical path in ALL stages passes through:

Y_reg (launch FF) -> Division/Modulus Logic (rem_96_35_Y_div_84_31) -> Output MUX -> out1_reg (capture FF)

The division logic is the bottleneck because integer division on 8-bit operands requires approximately 8 stages of subtract-compare-select operations, creating approximately 45-65 logic levels depending on the netlist optimization.